

POLITECNICO
MILANO 1863

DD: Design Document

Deliverable:	DD
Title:	Design Document
Authors:	Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo
Version:	1.0
Date:	7-January-2026
Download page:	https://github.com/nicolasbolanospolimi/BarberaBolanosNuzzo
Copyright:	Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Contents

Table of Contents . . . **3**

List of Figures . . . **4**

List of Tables . . . **4**

1 Introduction . . . **5**
1.1 Purpose . . . 5
1.2 Scope . . . 5
1.3 Definitions, Acronyms, Abbreviations . . . 5
1.4 Revision History . . . 5
1.5 Reference Documents . . . 5
1.6 Document Structure . . . 6

2 Architectural Design . . . **7**
2.1 Overview . . . 7
2.1.1 System View . . . 7
2.1.2 Detailed View . . . 8
2.2 Component View . . . 11
2.3 Deployment View . . . 13
2.4 Component Interfaces . . . 14
2.5 Runtime View . . . 15
2.6 Selected Architectural Styles and Patterns . . . 27
2.7 Other Design Decisions . . . 27

3 User Interface Design . . . **28**

4 Requirements Traceability . . . **35**
4.1 Sign up process . . . 35
4.2 Login and Logout . . . 36
4.3 Permissions . . . 37
4.4 Trip Management . . . 38
4.5 Bike Path Management- Manual mode . . . 40
4.6 Bike Path Management- Automated mode . . . 41
4.7 Finding the Best Path . . . 44

5 Implementation, Integration and Test Plan . . . **46**
5.1 Implementation Plan . . . 46
5.2 Integration strategy . . . 46
5.3 System testing . . . 46

6 Effort Spent . . . **49**

References . . . **50**

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

List of Figures

1 High level architecture. . . . 7
2 Detailed architecture. . . . 10
3 Component Diagram. . . . 11
4 Detailed Services Component Diagram. . . . 12
5 Deployment Diagram. . . . 13
6 Interfaces diagram for BBP. . . . 14
7 Register runtime diagram. . . . 15
8 Login runtime diagram. . . . 16
9 Search Paths runtime diagram. . . . 17
10 Record Trip runtime diagram. . . . 18
11 Attach weather information runtime diagram. . . . 19
12 Review trip runtime diagram. . . . 20
13 Enable automated mode runtime diagram. . . . 21
14 Disable automated mode runtime diagram. . . . 22
15 Create manual path runtime diagram. . . . 23
16 Record path with automated mode runtime diagram. . . . 24
17 Publish pre-recorded path in automated mode runtime diagram. . . . 25
18 Logout runtime diagram. . . . 26
19 Interaction between the pages regarding the sign up, log in and log out operations. . . . 28
20 Interaction between the pages regarding the manual path tracking operation. . . . 29
21 Interaction between the pages regarding the automated path tracking operation. . . . 30
22 Interaction between the pages regarding the recording and visualization of trips (personal activities). 32
23 Interaction between the pages regarding the search of possible paths. . . . 33
24 Navigation Bar links. . . . 34
25 Testing BBP Mobile Application . . . 47
26 Unit Testing . . . 47
27 Mocking services in order to test each cluster . . . 47
28 Mocking Search and Discovery Service . . . 48
29 Testing at infrastructure level the load balancer . . . 48

List of Tables

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

1 Introduction

1.1 Purpose

This document provides a comprehensive technical specification of the system previously defined in the Requirements Analysis and Specification Document (RASD). It evaluates the hardware and software architectures with particular emphasis on the modular interactions within the system. Additionally, the report details the methodologies for implementation, testing, and integration. Although the discourse is primarily technical and intended for developers, it serves as a valuable resource for stakeholders seeking insight into the system's developmental lifecycle.

1.2 Scope

This design document specifies the system's functional behavior across both standard and critical operational scenarios. It further defines the architectural framework by detailing the logical allocation of components and their respective interdependencies.

1.3 Definitions, Acronyms, Abbreviations

- RASD: Requirements Analysis and Specification Document
- DD: Design Document
- API: Application Programming Interface
- DBMS: Database Management System
- GDBMS: Graph Database Management System
- UML: Unified Modeling Language
- UI: User Interface
- IdP: Identity Provider

1.4 Revision History

- 07/01/2026 - Version 1.0
    + Added section 1
    + Added section 2
    + Added section 3
    + Added section 4
    + Added section 5

1.5 Reference Documents

The assignment for this document and all the information included refer to the following documentation:

- The specification for the 2025/26 Requirement Engineering and Design Project for the Software Engineering II course.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

- Requirement Analysis and Specification Document: "RASD"
- The slides on the webeep page of the Software Engineering II course.

1.6 Document Structure

Section 1

Offers a concise overview of the design document, outlining its purpose and scope, encompassing all the definitions, acronyms, and abbreviations employed.

Section 2

Extensively explores the system architecture, offering an intricate depiction of the components, interfaces, and all the technical decisions undertaken for application development. It encompasses detailed sequence, component, and ArchiMate diagrams providing an in-depth understanding of the system.

Section 3

Provides a comprehensive portrayal of the UI, including all client side mockups alongside with graphs that explains the proper execution flow and detailed explanations.

Section 4

Maps the goals and requirements outlined in the RASD with the functionalities and components presented in this document.

Section 5

Presents the implementation, testing, and integration phases of the system components, which will be undertaken during the technical development of the application.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2 Architectural Design

2.1 Overview

This section provides a high-level overview of the system architecture, detailing its main components and the interactions among architectural layers. It also describes the modeling decisions made to satisfy both the functional and non-functional requirements defined in the RASD document.

2.1.1 System View

The BBP system adopts a microservices-based architecture [5] to support the scalable and independent evolution of user management, trip tracking, and bike-path management functionalities. The architecture is organized into several tiers.

1. **The mobile application tier** acts as the presentation layer, providing all user interfaces.
2. **The API gateway tier** receives client requests and routes them to the appropriate microservices.
3. **The microservices tier** contains the set of services that implement BBP's core business capabilities.
4. **The data tier** ensures that each service owns and accesses its dedicated storage.
5. **The external services tier** represents the external APIs and systems consumed by BBP.

This architectural style improves fault isolation and allows each service to be deployed, scaled, and maintained independently as usage patterns evolve, for example when trip recording or path search become more demanding than user registration.

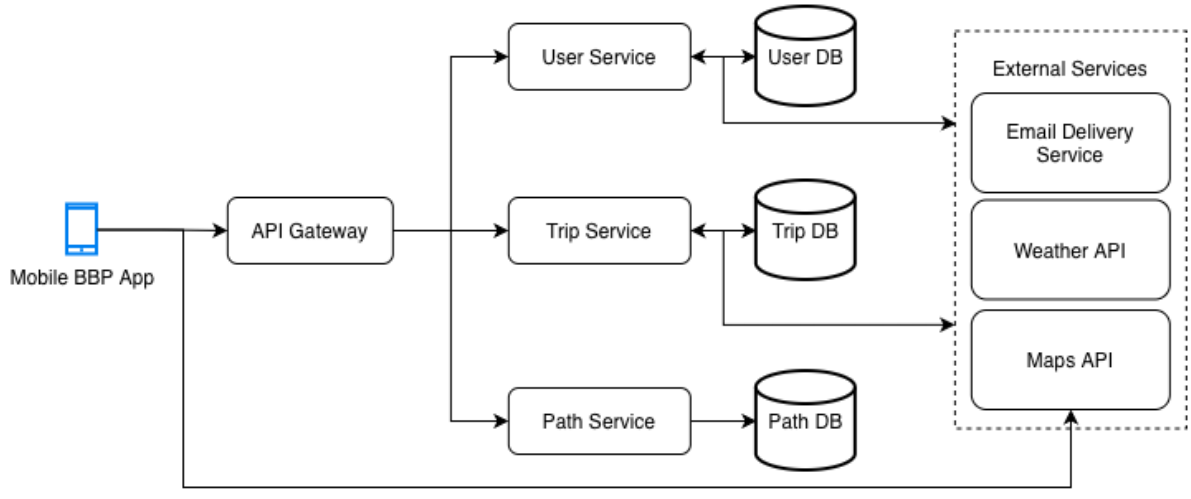


Figure 1: High level architecture.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2.1.2 Detailed View

Mobile application tier

The Mobile BBP App is the primary client used by cyclists to register, record, and browse trips and bike paths while riding. It interacts with the backend over HTTPS, sending GPS, sensor data, and user inputs (manual path insertion, confirmation of potholes) required by the BBP domain. This client keeps only presentation and local cache, delegating computation such as statistics, scoring, and path merging to backend services to preserve consistency.

API gateway tier

The API Gateway acts as the single entry point to the microservices landscape. It routes incoming calls to the appropriate service (User, Trip, or Path), performs authorization, applies rate limiting, and can aggregate responses so that the mobile application does not need to orchestrate multiple calls for a single screen.

The gateway integrates a built-in API authorizer, which validates client credentials and access tokens, evaluates access policies, and enforces fine-grained permissions before forwarding a request to any backend service. This ensures that only authenticated and authorized users can invoke protected operations, while keeping security logic centralized and consistent across the system.

The API Gateway abstraction also hides internal service URLs, protocols, and versioning, which simplifies the mobile client and allows services to evolve, be replaced, or be scaled independently without breaking existing applications.

Microservices tier

This tier host all the business logic of the application, separated by services. The User Service manages registration, authentication, and user profiles, supporting all features that require identifying the user. The Trip Service handles the creation, storage, and analysis of trip data. It also enriches trips with weather information retrieved from the external Weather API when available. The Path Service manages bike-path entities, their status, and obstacle reports, and is responsible for merging publishable information from multiple users over time to derive consolidated path scores used for route visualization.

Adopting a microservices architectural style provides several advantages. Each service encapsulates a clearly defined set of responsibilities and owns its data, which reduces coupling and enables independent development, deployment, and scaling. For example, the Trip Service can be scaled horizontally to handle high-volume sensor data without affecting the User or Path Services. Fault isolation is also improved, since a failure or performance degradation in one service (such as the integration with the Weather API) does not necessarily compromise the availability of the others. Furthermore, this decomposition allows teams to evolve the technology stack or data models of a single service with minimal impact on the rest of the system.

The architecture is designed with observability in mind. Each microservice exposes metrics, logs, and traces that can be collected and correlated by centralized monitoring and logging tools. Structured application logs, domain-specific metrics (e.g., number of trips processed, average trip enrichment latency), and distributed tracing across User, Trip, and Path Service calls enable effective end-to-end monitoring of user workflows. This observability layer facilitates proactive detection of anomalies, faster diagnosis of incidents, and data-driven capacity planning, thereby improving the reliability and maintainability of the BBP platform over time.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Data tier and External services tier

Each microservice owns a dedicated database: User DB, Trip DB, and Path DB. Ensuring loose coupling and independent scaling of storage according to access patterns (for instance, high-write trip data versus relatively stable user records). This database-per-service pattern also supports autonomy in schema evolution and technology choice, which is valuable as BBP analytics or geospatial needs grow.

Also, dedicated integrations with the Email Delivery Service and Weather API allow services to send notifications and enrich trips with meteorological information without exposing external APIs directly to the mobile client. Isolating these integrations inside backend services keeps credentials and error handling on the server side, and shields users from third-party outages through backend-level resilience strategies.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

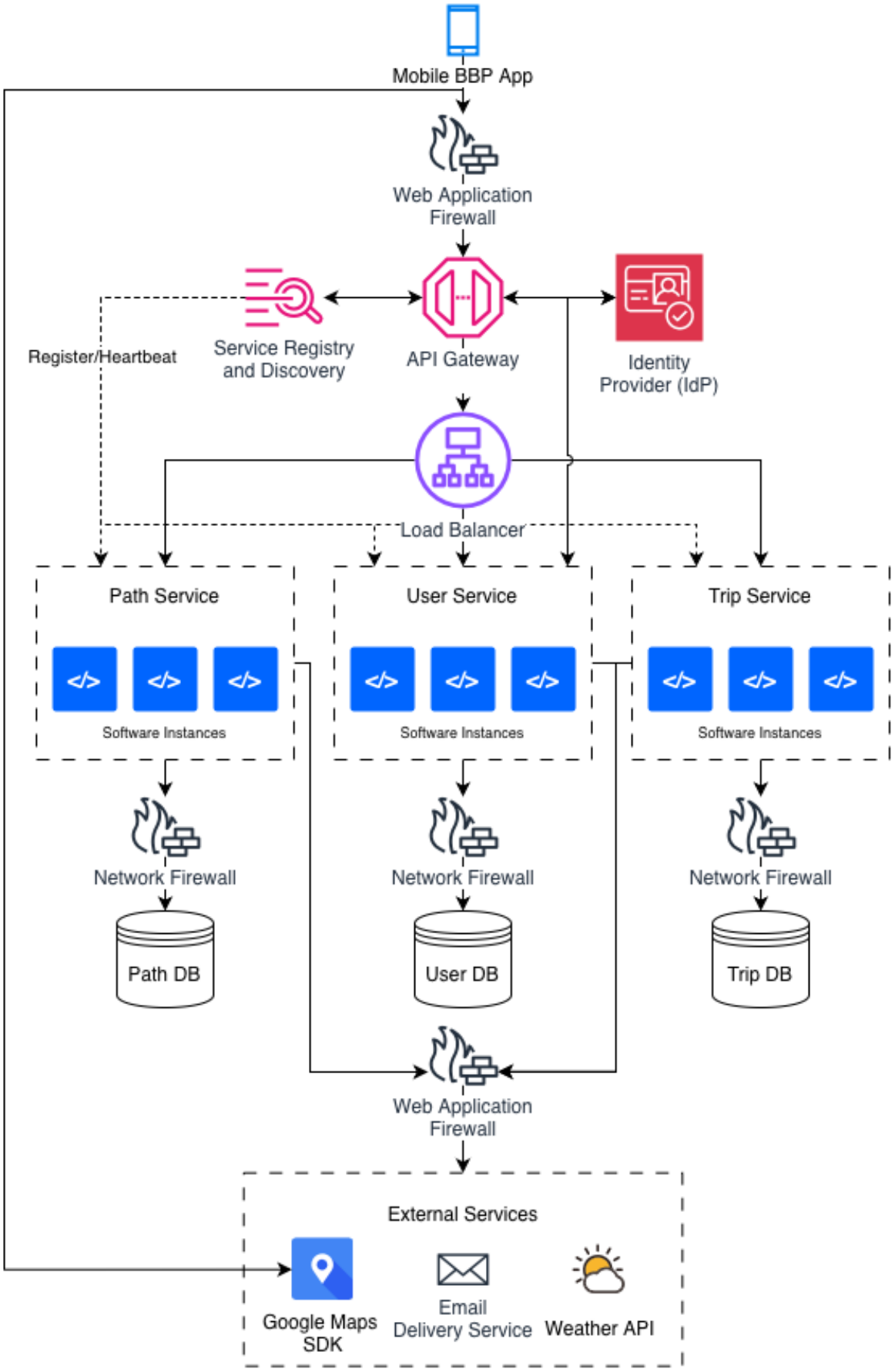


Figure 2: Detailed architecture.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2.2 Component View

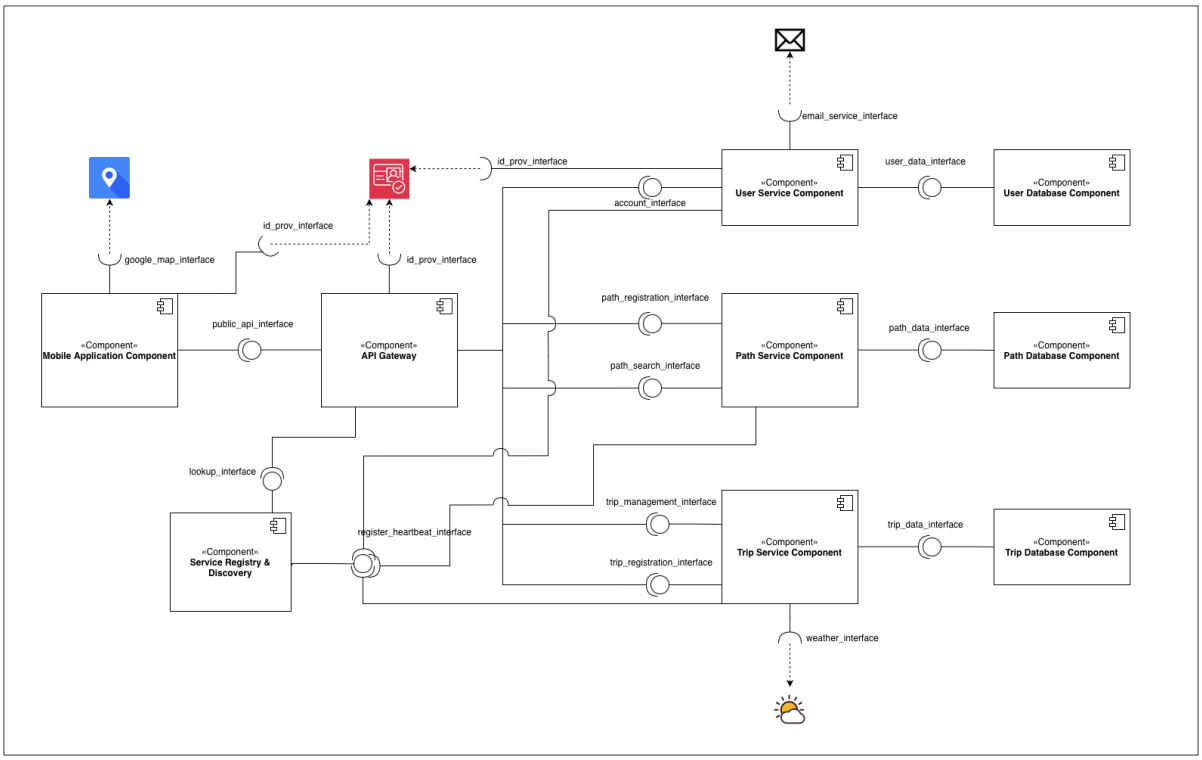


Figure 3: Component Diagram.

This diagram provides a high-level representation of the main components of the BBP system, where the application layer is modeled through the API Gateway and microservice components, while their internal structure is detailed in subsequent sections.

The interfaces depicted on the left correspond to the interactions between the Mobile Application Component and the backend entry point, and capture the principal functionalities requested by clients, such as user management, trip registration, and path search.

On the right side, the diagram highlights the external interfaces consumed by the backend, including the Email Delivery Service and Weather API, as well as the dedicated database components that expose data-access interfaces for user, path, and trip information, all managed by the respective service components to satisfy the system's persistence and integration needs.

Details regarding the microservices components are presented on figure 4 alongside the respective description.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

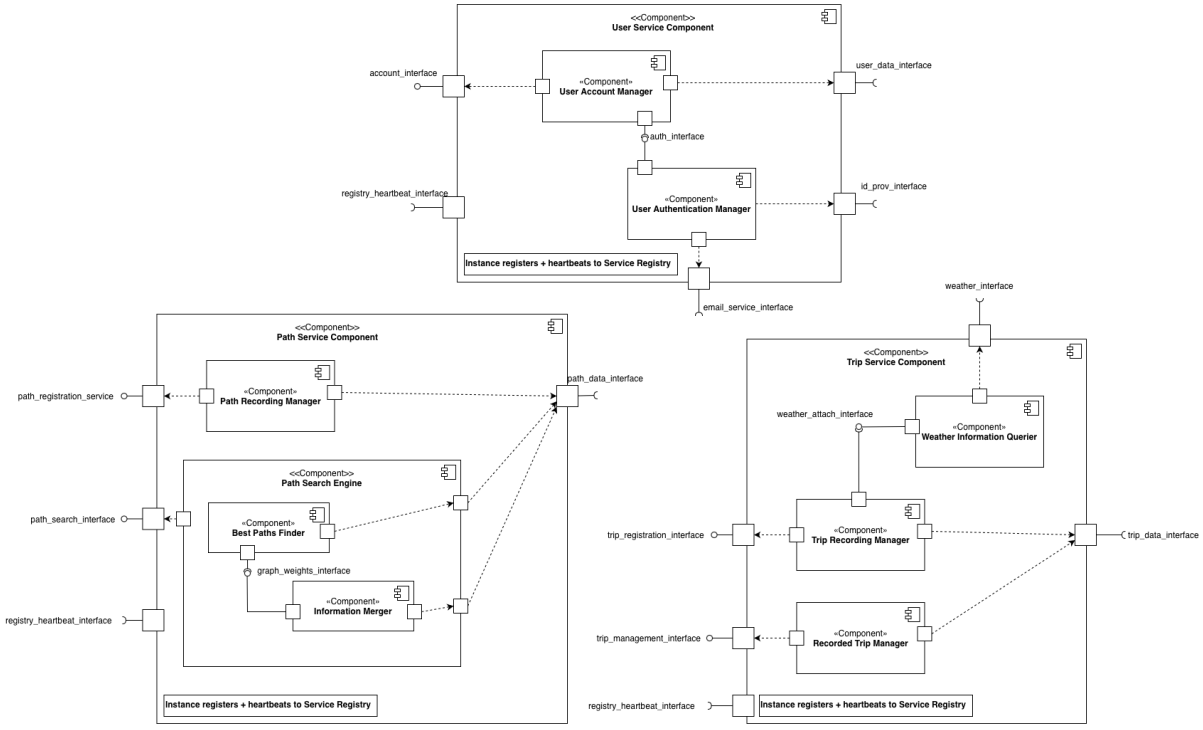


Figure 4: Detailed Services Component Diagram.

User Service Component

- **User Account Manager** handles core life-cycle operations for user accounts, including sign-up (user registration), login, account activation/deactivation, and deletion. This sub-component enforces business rules for account state transitions.
- **User Authentication Manager** manages secure access control, supporting operations such as sign-up validation, login token issuance, account confirmation, password reset flows, and sign-out. It delegates identity verification to the external Identity Provider and integrates with the Email Delivery Service for confirmation emails.

Trip Service Component

- **Weather Information Querier** retrieves meteorological data for recorded trips by invoking the external Weather API with trip timestamps and geolocations. This sub-component implements resilience patterns (timeouts, retries) and caches recent weather data to minimize external dependencies and support offline trip enrichment.
- **Trip Recording Manager** orchestrates the creation of new trips from mobile GPS/sensor data, invoking `CreateTrip` to persist raw trip information and scheduling `CreateWeatherJob` to asynchronously fetch and associate weather data via the Weather Information Querier.
- **Recorded Trip Manager** provides query interfaces for trip retrieval, supporting `GetTrips` (list user's trips with filtering/sorting) and `GetTrip` (retrieve detailed metrics like distance, average speed, and weather for a specific trip ID).

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Path Service Component

- **Path Recording Manager** manages the lifecycle of bike path entities, implementing `CreatePath` (manual/automated path registration), `VerifyPath` (user confirmation/correction of detected obstacles like potholes), and `PublishPath` (promotion of verified paths to community-visible status).
- **Path Search Engine** computes optimal bike routes between origin/destination points by coordinating path discovery and scoring.
  - **Best Paths Finder** implements `ComputePossiblePaths`, evaluating candidate routes based on path scores, distance effectiveness, and user preferences while integrating geospatial queries via Maps SDK.
  - **Information Merger** aggregates publishable path data from multiple users, applying freshness weighting and confirmation counts to resolve conflicts (e.g., prioritizing "optimal" status from multiple recent reports over older "requires maintenance" tags).

2.3 Deployment View

The deployment diagram shows BBP components distributed across client devices (Mobile App), security nodes (WAF, API Gateway), container hosts (microservices with load balancers), databases (Path/User/Trip DBs), and external providers (Google Maps, Email, Weather API).

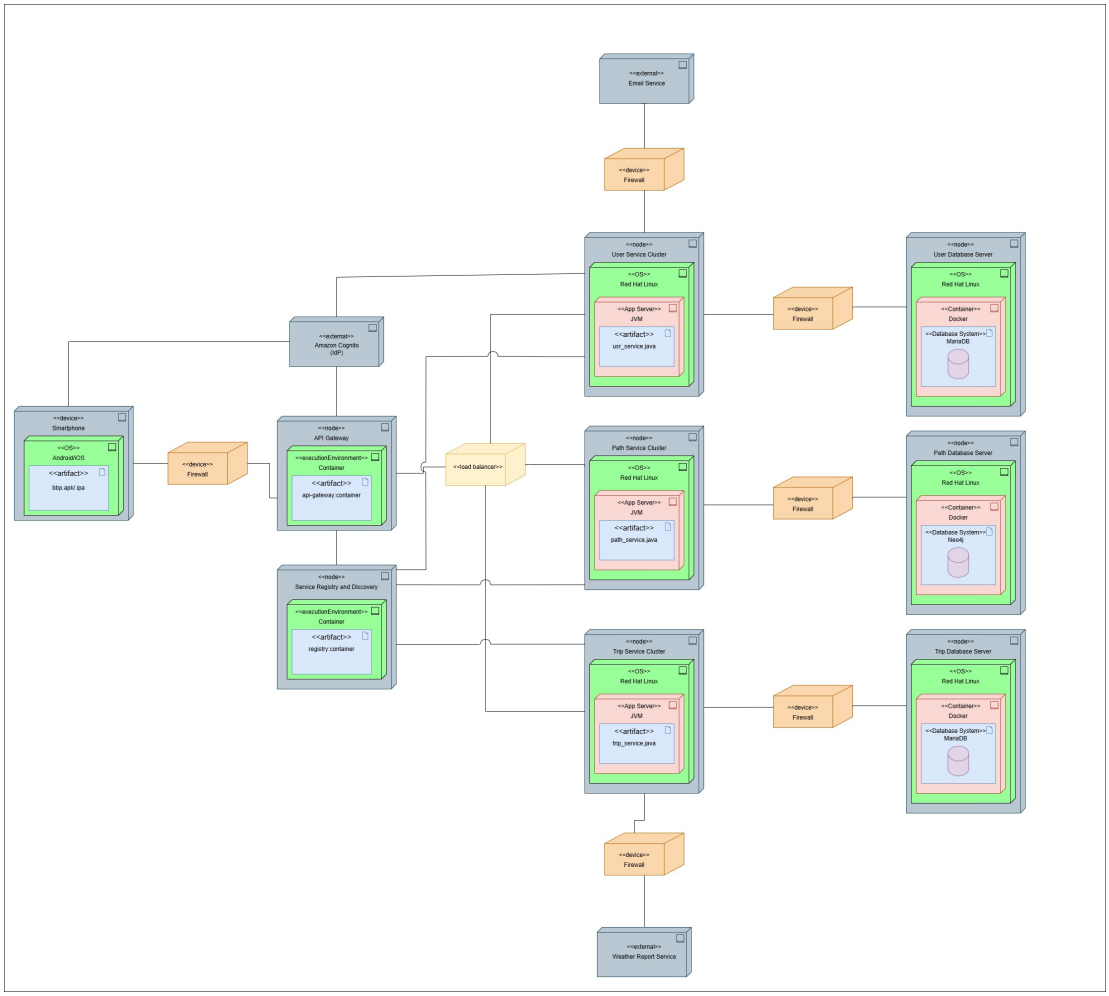


Figure 5: Deployment Diagram.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

The diagram presents the relationships and the flow of data among the following components.

- **Smartphone**: Hosts the Mobile BBP App, executing client-side presentation logic, GPS/sensor data capture, and map rendering via Google Maps SDK. Here the data is captured and sent to the backend services.
- **API Gateway**: Single entry point for client requests, handling routing, authorization (via IdP), rate limiting, and service discovery integration.
- **Service Registry**: Centralized directory for dynamic service registration and discovery, enabling the API Gateway to locate available microservice instances. Also responsible for checking the health of instances via heartbeat. [2]
- **User/Path/Trip Cluster**: Server orchestration groups hosting multiple instances of User, Path, and Trip Services for scalability and high availability.
- **User/Path/Trip Database**: Dedicated, per-service relational or NoSQL databases storing user profiles, path metadata/obstacles, and trip records/metrics respectively.
- **External Components**: Third-party services including Google Maps (geospatial), Email Delivery (notifications), and Weather API (trip enrichment), accessed via secure outbound connections.

2.4 Component Interfaces

The following diagram provides a comprehensive high level depiction of the interfaces and their associated methods offered by each component within the BBP platform. It is essential to acknowledge that the delineated methods do not precisely mirror the final version intended for implementation; rather, they serve as a logical representation

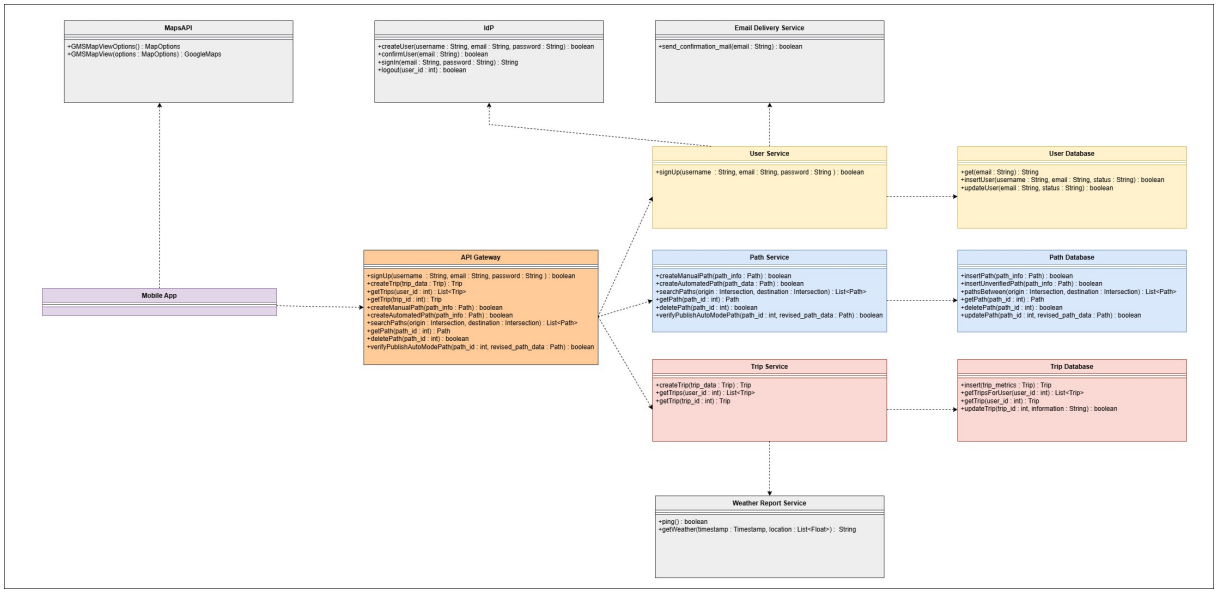


Figure 6: Interfaces diagram for BBP.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2.5 Runtime View

This section presents several practical use cases of the system, illustrated by sequence diagrams.

Register

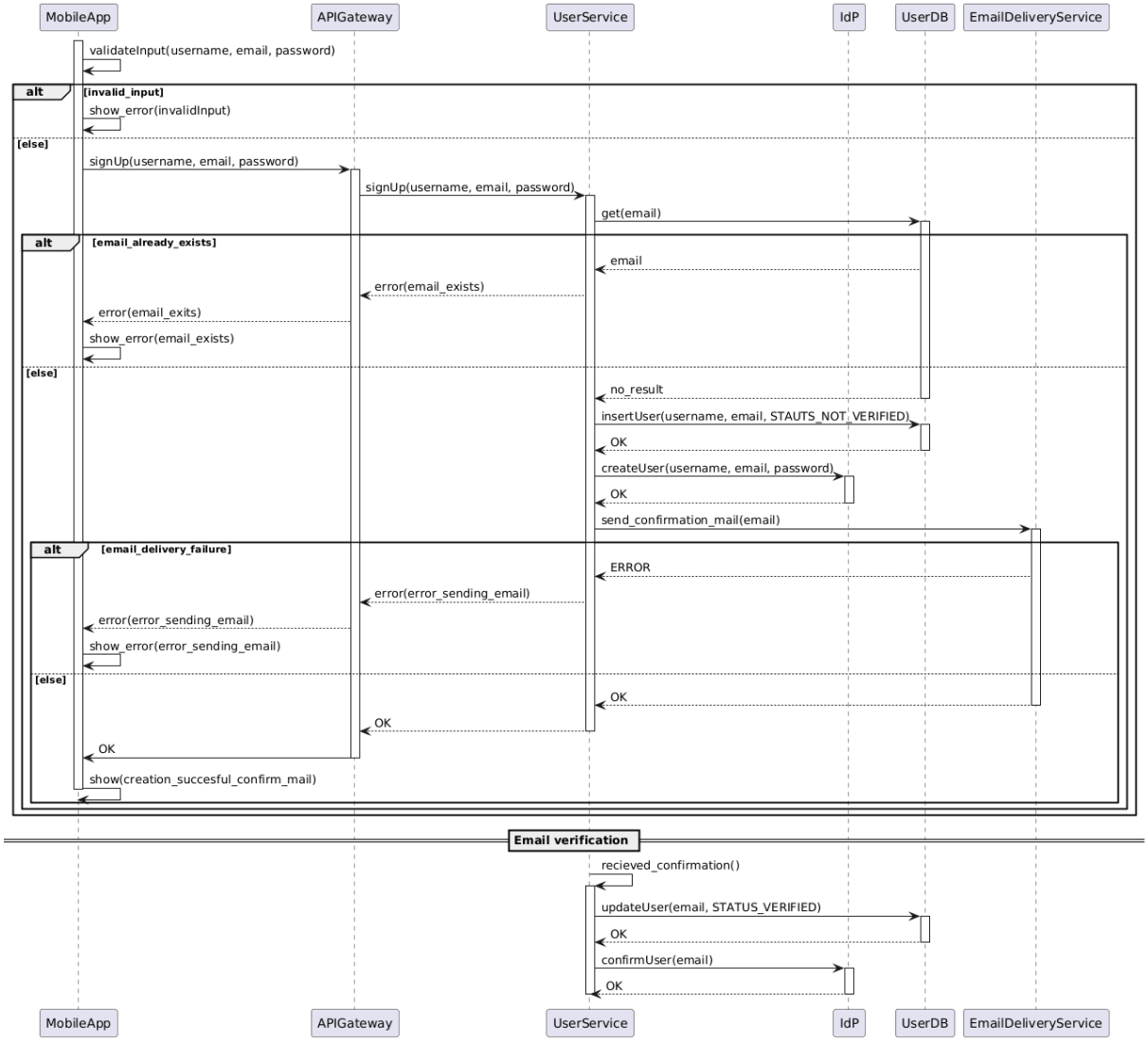


Figure 7: Register runtime diagram.

The *Sign-Up* phase shows the Mobile App submitting user details via API Gateway to User Service, which checks for email uniqueness in User DB. Then, it registers the user into the IdP and sends a confirmation account email. Once there is user confirmation for account completion the user service updates the user in the service database and IdP.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Login

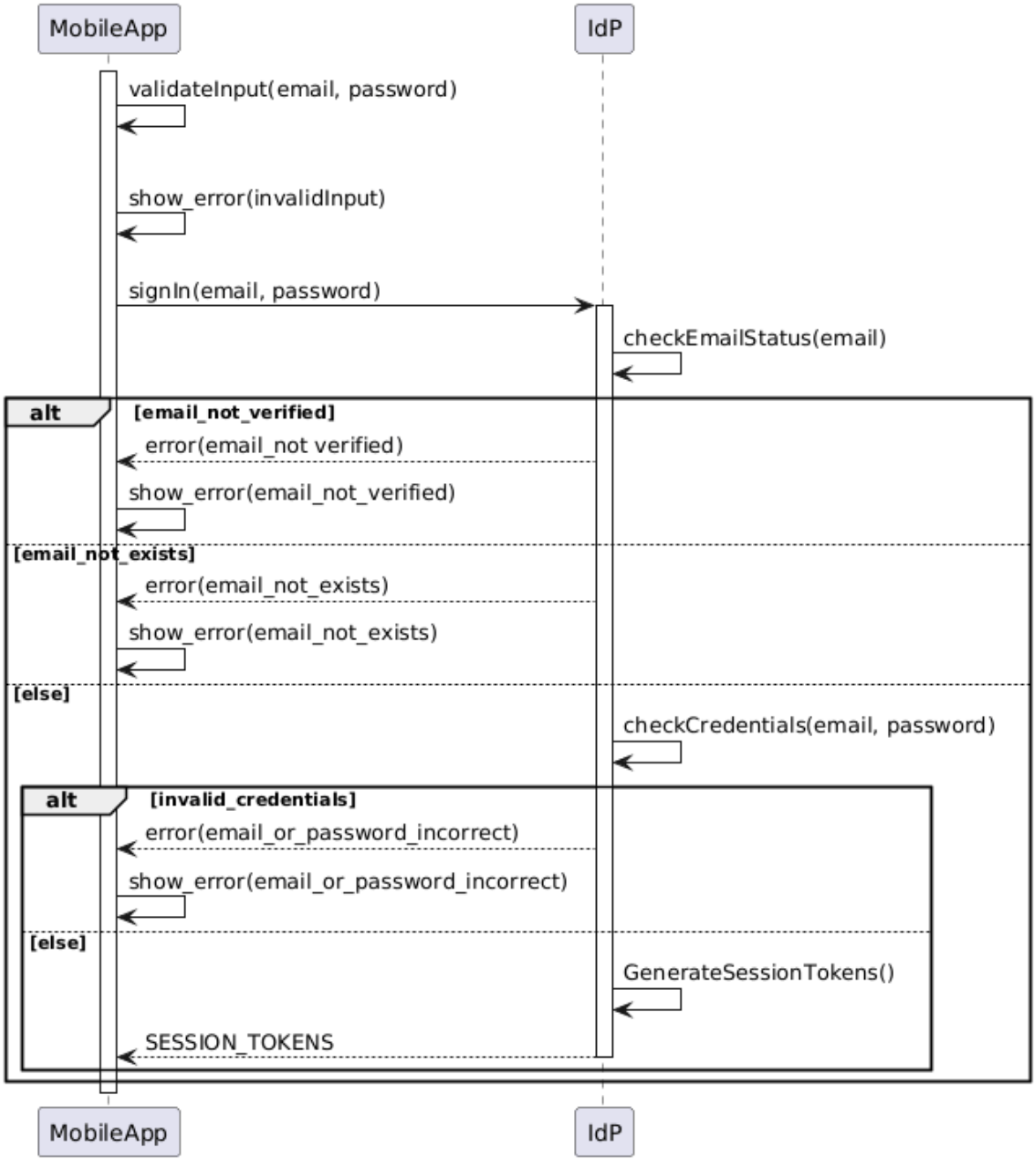


Figure 8: Login runtime diagram.

The *Login* phase shows the Mobile App submits credentials to the IdP, which first verifies user existence and account status, then authenticates the credentials. Upon success, session tokens are issued; otherwise, an appropriate error is returned.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Search Paths

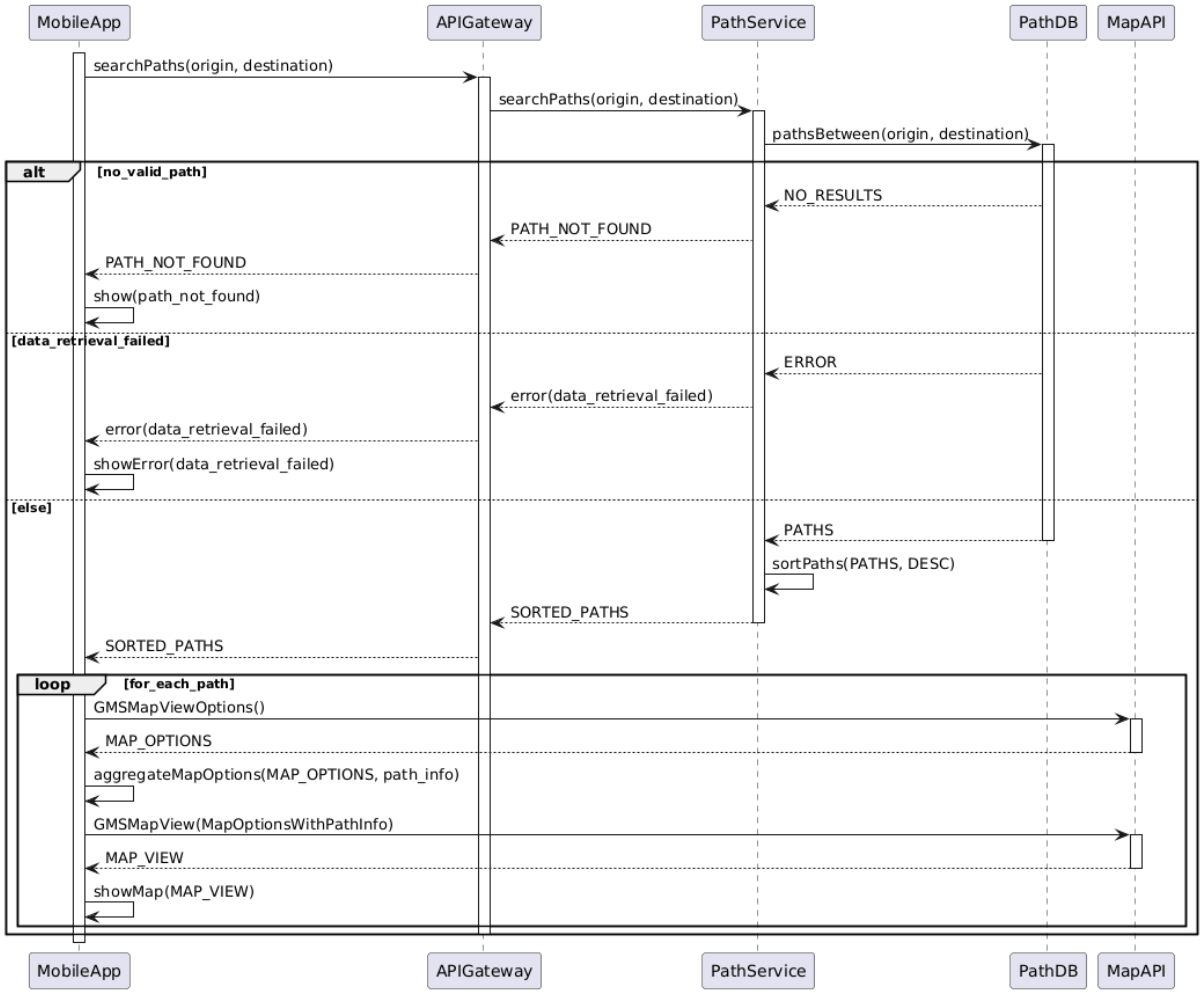


Figure 9: Search Paths runtime diagram.

The *Search Path* phase shows the Mobile App sends origin and destination via API Gateway to Path Service, which queries Path DB for matching bike paths. Results may include zero, one, or multiple paths based on available data.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Record Trip

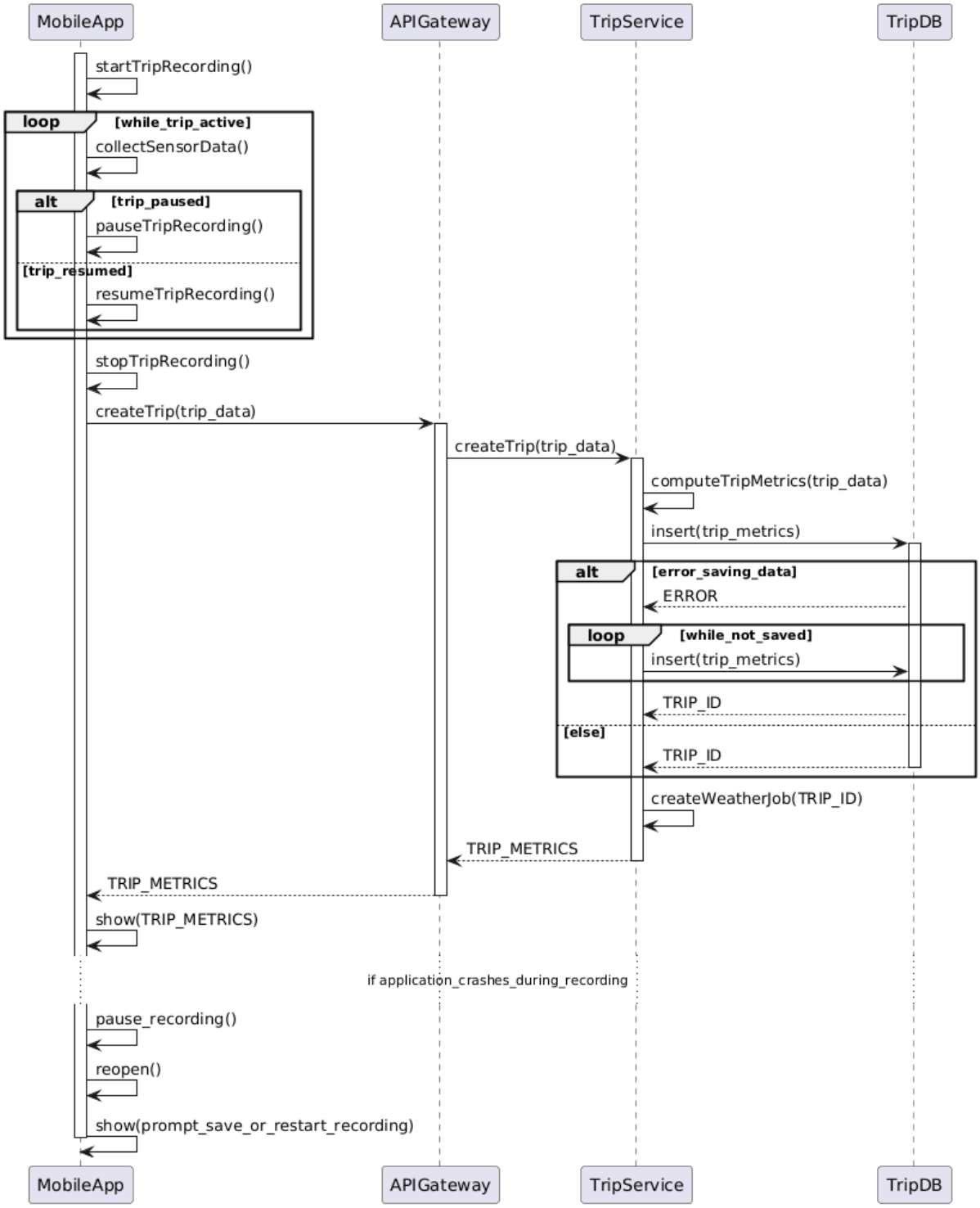


Figure 10: Record Trip runtime diagram.

The *Record Trip* phase shows the Mobile App gathers trip data from the device sensors. Data is then sent to Trip Service via API. Trip Service creates trip records in Trip DB, computes metrics, and schedules weather enrichment jobs.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Attach Weather Information

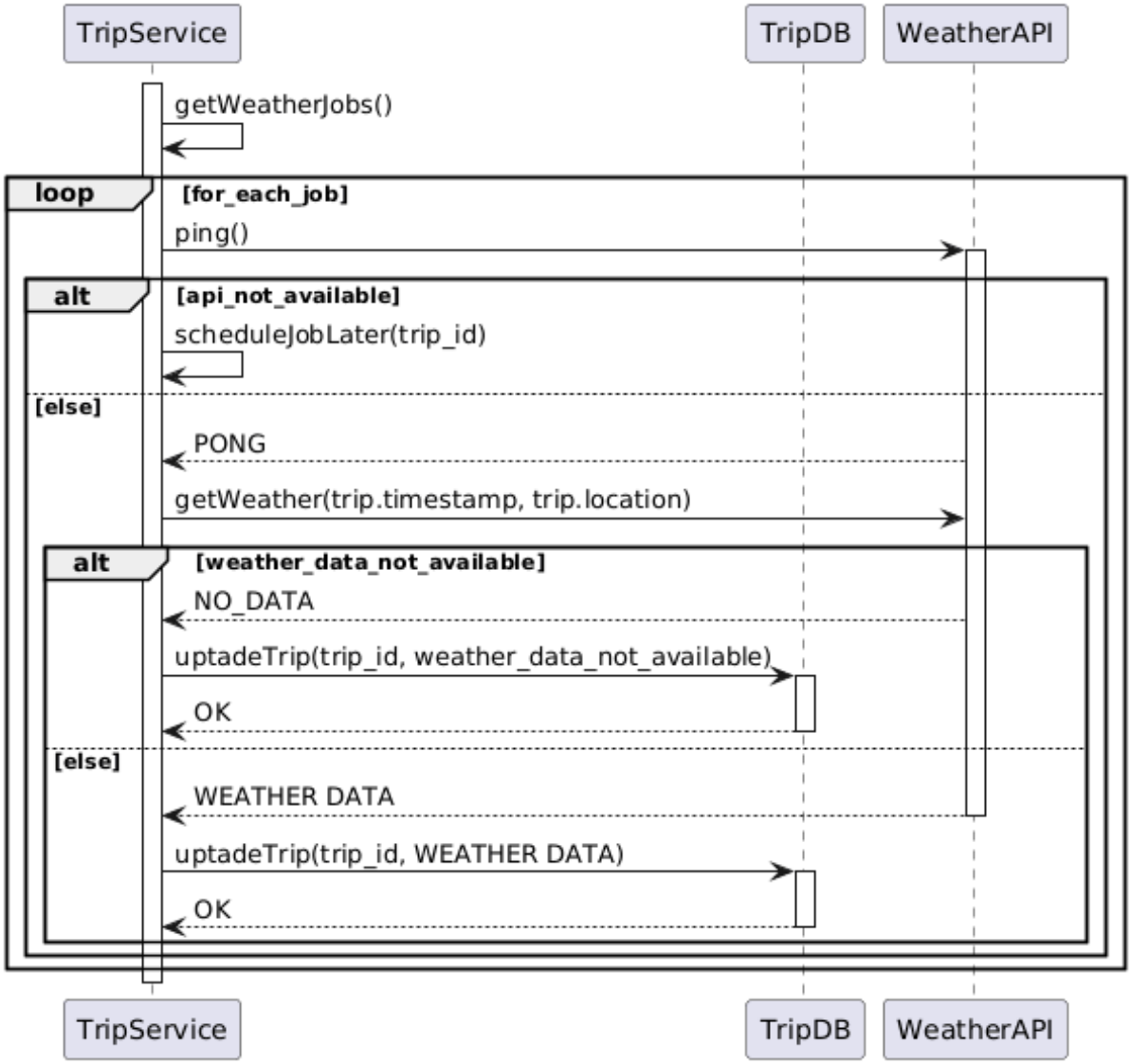


Figure 11: Attach weather information runtime diagram.

The *Attach weather information* phase shows the Trip Service listing the scheduled weather jobs. For each it verifies the availability of the weather API, then making the request for weather information. It finally attaches the information to the corresponding trip in the service database.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Review Trip

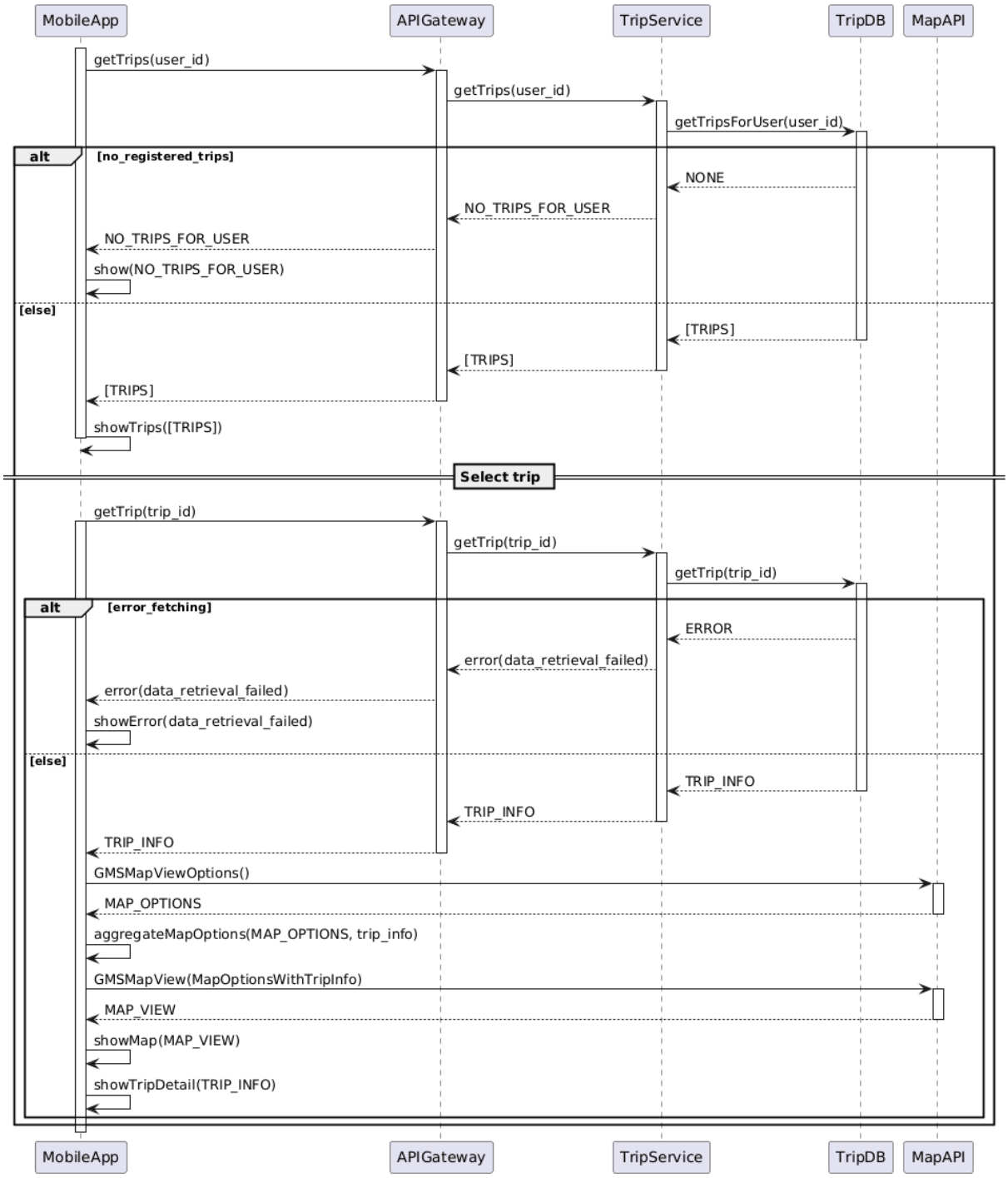


Figure 12: Review trip runtime diagram.

The *Review Trip* phase shows Mobile App requesting the list of trips for a user to the Trip Service via API gateway. Either none are returned or a list of trips. Then one trip is selected and is requested from the mobile app, the service then returns the corresponding information and gets displayed.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Enable Automated Mode

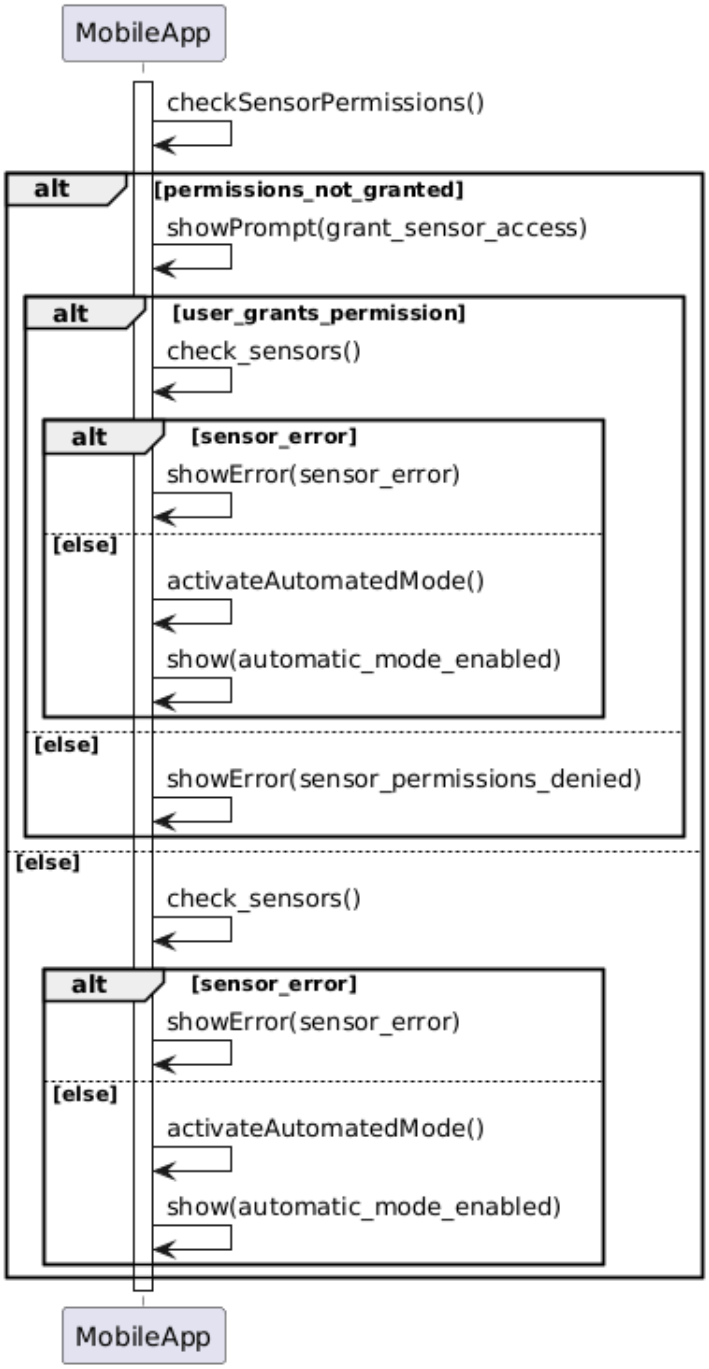


Figure 13: Enable automated mode runtime diagram.

The *Enable automated mode* phase shows Mobile App checking sensor permissions. If not granted, it prompts the user to grant access. If granted, the app checks sensors availability and starts gathering data in the background.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Disable Automated Mode

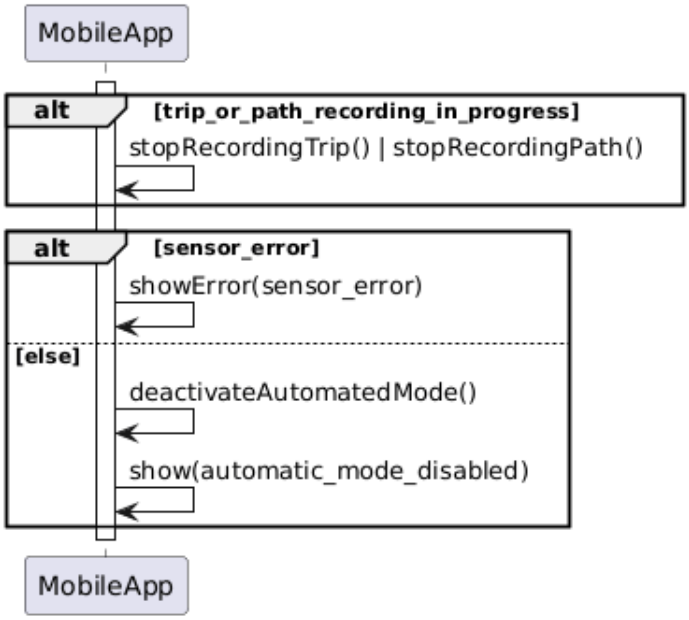


Figure 14: Disable automated mode runtime diagram.

The *Disable automated mode* phase shows Mobile App checking if a trip or path recording is in progress, if yes then it stops the recording. Then it stops gathering sensor data and stops background execution.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Create Manual Path

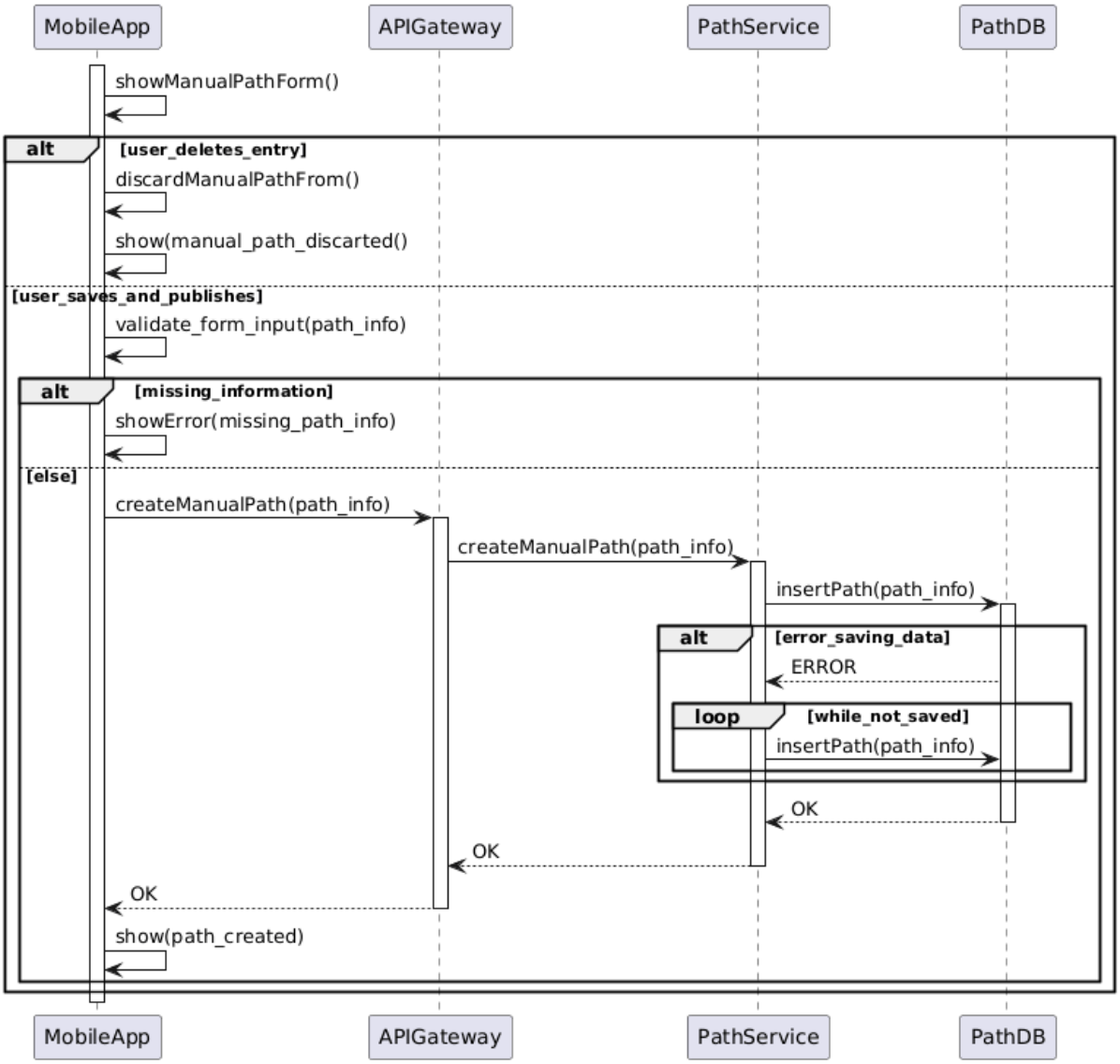


Figure 15: Create manual path runtime diagram.

The *Create manual path* phase shows Mobile App showing the manual path form, then the user fill in the information. The user can decide whether discard the form or publish the path. If the decision is to publish the Mobile App validates the form information and sends it to the Path service via API gateway. The Path service saves this information in the service database.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Record Path with Automated Mode

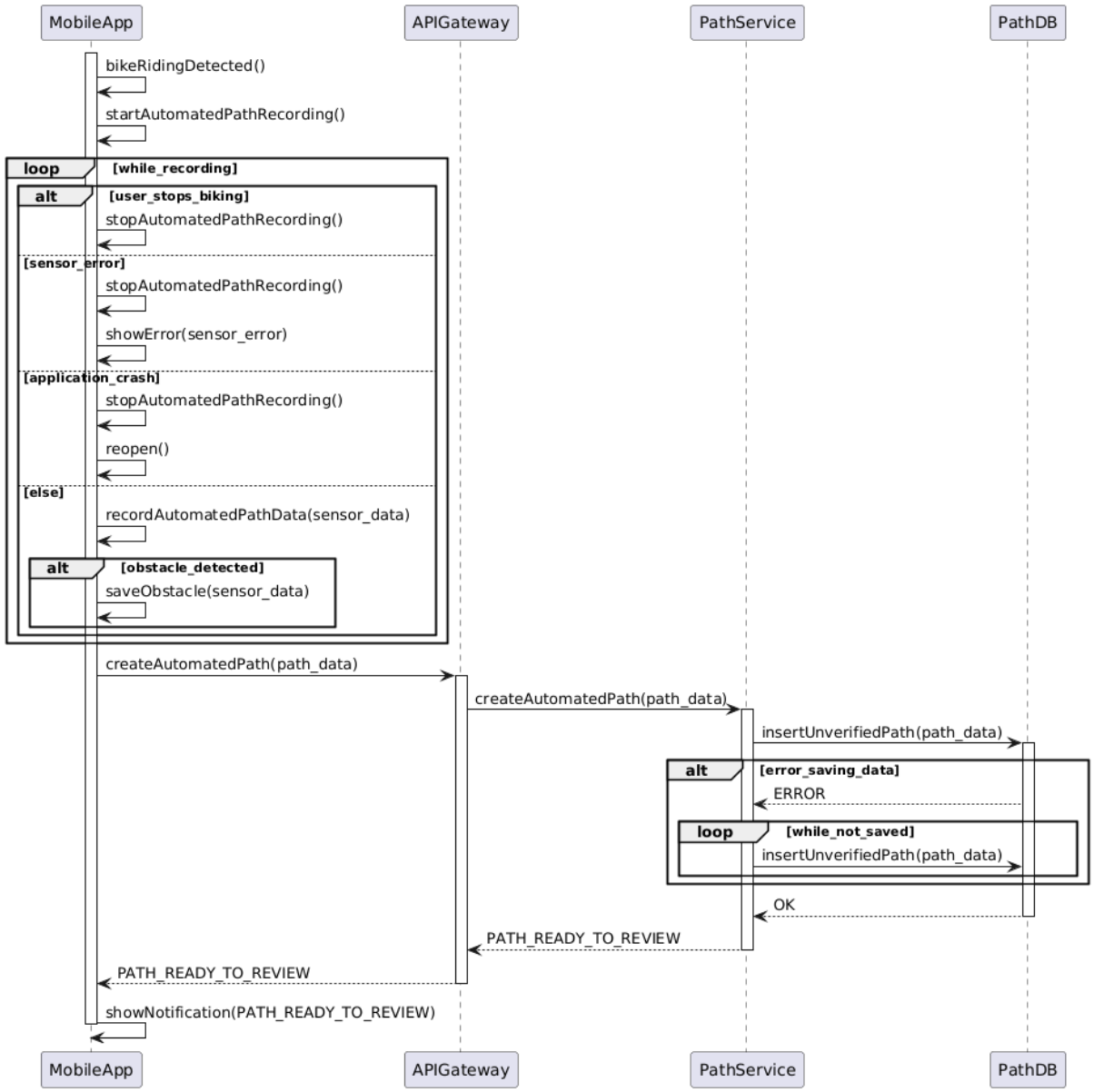


Figure 16: Record path with automated mode runtime diagram.

The *Record path with automated mode* phase shows Mobile App detecting a bike ride and start a path recording. While the user rides the bike the Mobile app records data from the sensors and detects obstacles. It stops the recording if the user stops riding, a sensor error occurs or the application crashes. It then sends the collected data to the Path service via API gateway. The services storage this data on the service data base and marks it as unverified for later user revision.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Publish Pre-recorded Path in Automated Mode

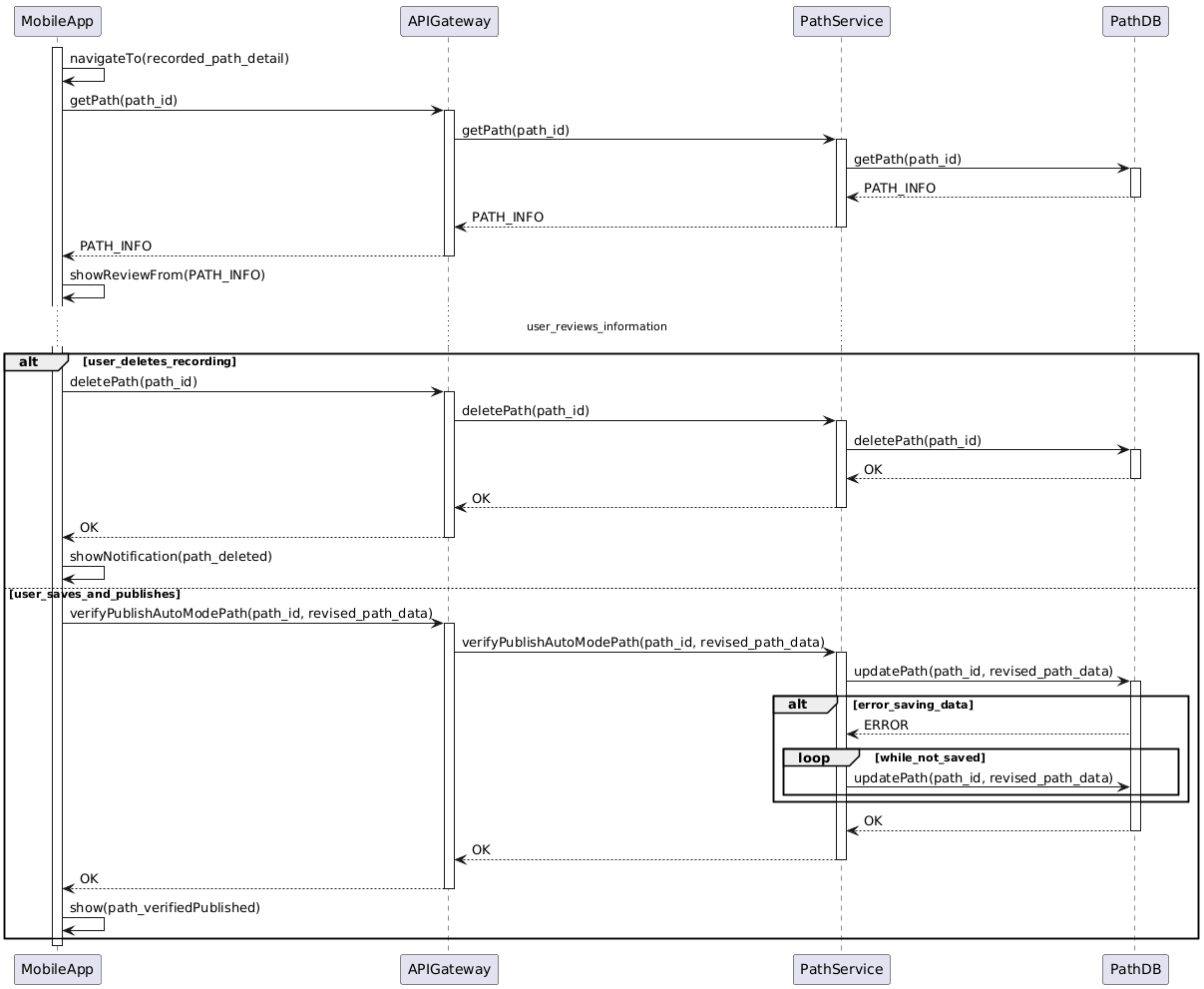


Figure 17: Publish pre-recorded path in automated mode runtime diagram.

The *Publish pre-recorded path in automated mode* phase shows Mobile App navigating to the detail of path pending review. Then the user reviews and can decide whether to discard it or publish it. In case of discard, the mobile app sends a deletion request to the path service and the path gets eliminated from the service database. Otherwise the mobile app sends a request to the path service to update the path with the revised information.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Logout

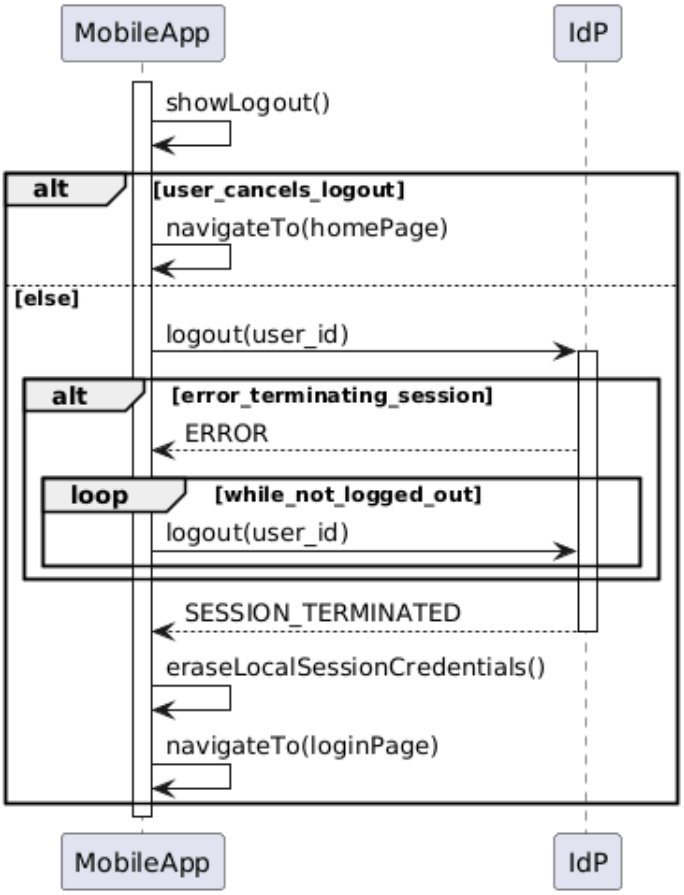


Figure 18: Logout runtime diagram.

The *Logout* phase shows Mobile App showing the user the logout dialog. The user can either cancel or confirm the logout action. In case of confirmation, the mobile app erases local cache, requests the IdP to terminate session credentials and navigates to the login page.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

2.6 Selected Architectural Styles and Patterns

Here below are discussed the utilized architectural styles and patterns:

- **Microservices architecture**: the chosen system architecture is a microservices architecture, being the natural approach for this type of application. In this case, as a matter of fact, the single main features (User, Trip and Path management) are easily distinguishable and with a very separated runtime, making the microservices architecture the best choice. In this approach, every feature is run by a different component (or set of components) specialized for a single task. The application logic is deliberately decoupled from the data tier to ensure a clear separation of concerns and to promote specialization..
  Main advantages:
  - Improved resilience
  - Fine-grained scaling
  - Easier deployment
  - Technology-neutral protocols (REST APIs, used for communication by all of the microservices)
  - Concurrency
- **Security pattern: API Gateway [4]**: in a microservice architecture, an API Gateway is essential. It is a centralized entry point that manages and routes requests from clients to the correct microservice. Introducing caching at this level may help latency.
  Main advantages:
  - Single gateway for clients
  - Routing and Load balancing
  - Authentication and Authorization methods

  The main challenges to consider are:
  - Possible performance bottleneck and single point of failure: to make sure they can support the load, different instances of the gateway are necessary, with also adding load balancing if needed;
  - Scalability: consider horizontal scalability and load balancing
  - Security risks (if not configured correctly)
- **Routing pattern: Service Registry and Discovery [3]**: to guarantee service discovery and reachability, a service registry and discovery layer is a must have in this architecture. The API Gateway looks up for available services using the nodes in this layer. The single service instances will register to this layer, and notify them of their availability using a heartbeat signal.
  Main advantages:
  - More efficient load balancing
  - Fault tolerance (service discovery can route traffic to other healthy instances)
  - Dynamic environment, without manual intervention
  - Easier scalability

2.7 Other Design Decisions

- **Security: Amazon Cognito (IdP) [1]**: to guarantee a better authentication mechanism, Amazon Cognito is a great choice. It can be used to create "pools" of users with a predetermined set of permissions, that acquire a specific authorization token (JWT) to pass to the API Gateway. In this way, it is possible to identify the groups of users and their permissions.
- **GDBMS**: using a Graph Database Management System seemed to suit better the information stored in the path database. In fact, it is easier to perform queries, if data is structured as nodes (intersections) and edges (roads), thanks to the better overlap between reality and model.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

3 User Interface Design

The main objective of this UI is to make the user experience as clear and intuitive as possible. The interface is designed for a mobile app, focusing on essential information, clean layouts, and straightforward interactions to help users achieve their goals quickly and effortlessly.

Sign Up, Log In and Log out

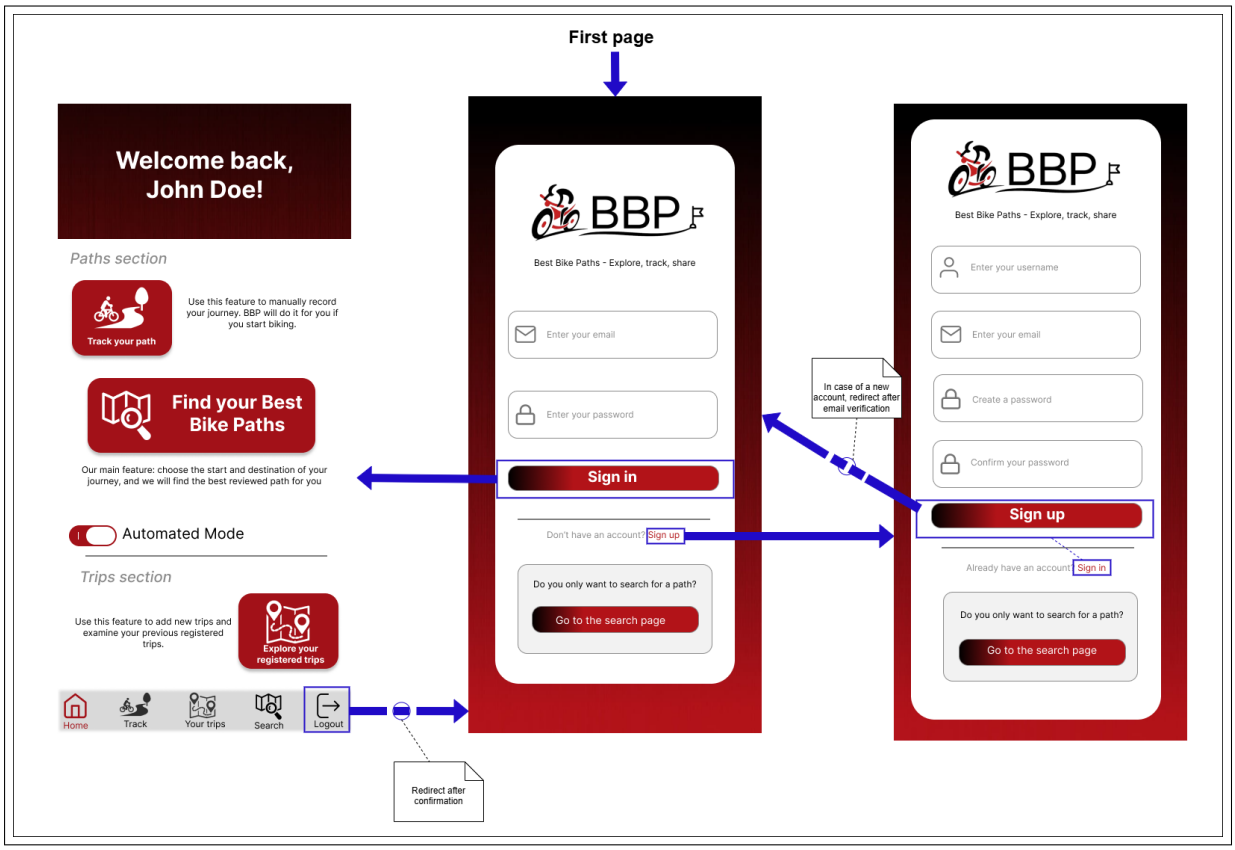


Figure 19: Interaction between the pages regarding the sign up, log in and log out operations.

When the user opens the application, the Log In page is loaded. Here, the user can enter the system by providing their credentials and by pressing the "Sign In" button.
If the user wants to register a new account, they can be redirected to the Sign Up page by pressing the "Sign up" text-button in the Log In page. Here the user can insert their information and, after an email verification process (not shown here for simplicity), successfully register into the system. After the process is completed, they are automatically redirected to the Log In page for their first login operation.
If the user wants to delete their session data, they can log out from the system by pressing the "Logout" button at the bottom of the screen, where the Navigation Bar is located. They will be redirected to the Log In page.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Manual path tracking

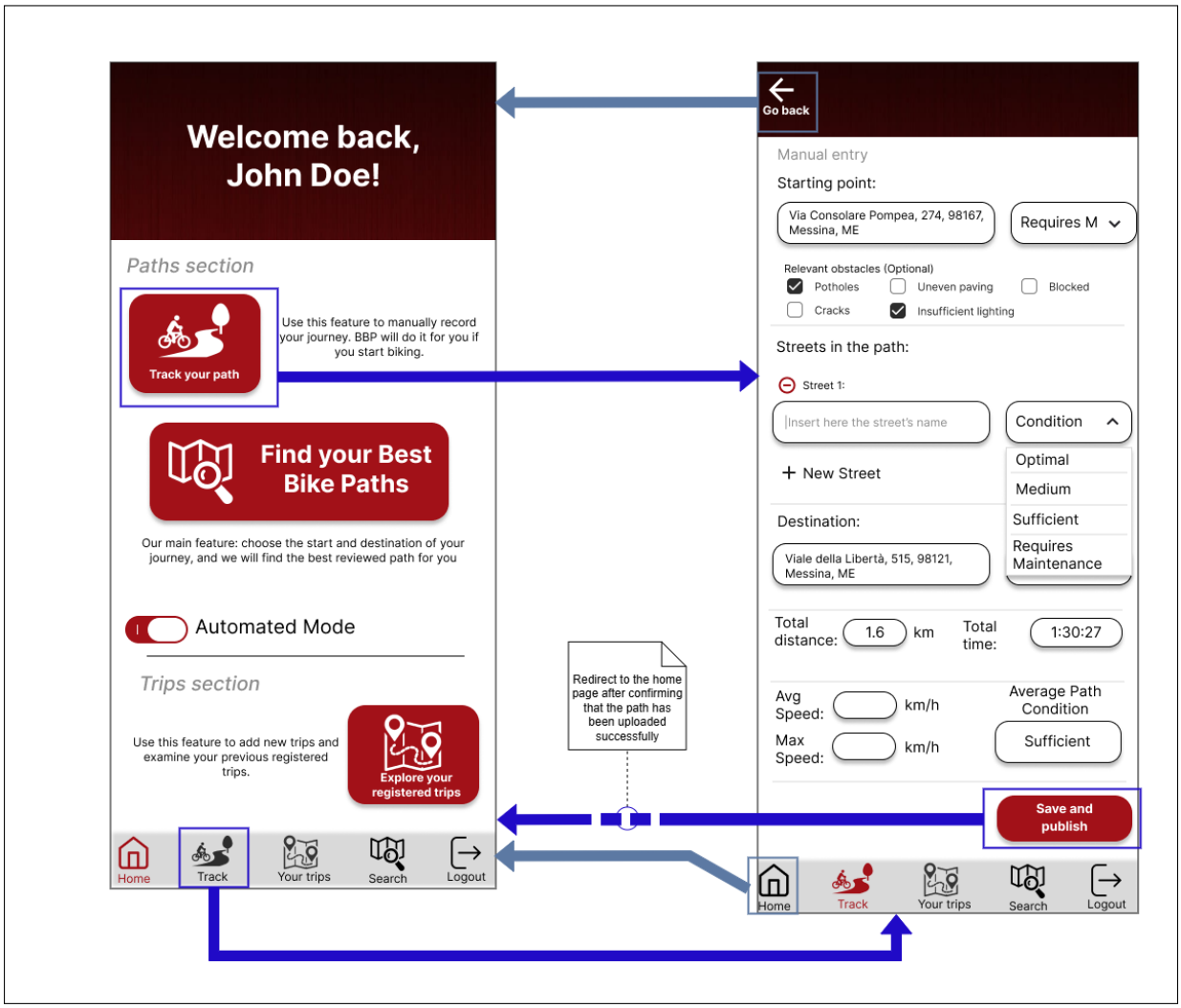


Figure 20: Interaction between the pages regarding the manual path tracking operation.

A registered user can easily access the manual path tracking by pressing the corresponding button in the home page or by pressing the track icon in the Navigation Bar, at the bottom of every page.
Here the registered user must insert the starting and destination points of their bike path, and can, at any time, insert/remove another road between these two points by pressing the corresponding buttons.
When the path needs to be registered into the system, the registered user can press the "Save and publish" button, successfully completing the operation. A notification is displayed to the registered user confirming the completion of the operation (not shown here for simplicity) and they are automatically redirected to the home page.
At any point of the recording, the registered user can abort the procedure (and consequently clearing the fields) by pressing the "Go back" button at the top or the Home icon in the Navigation Bar.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Automated path tracking

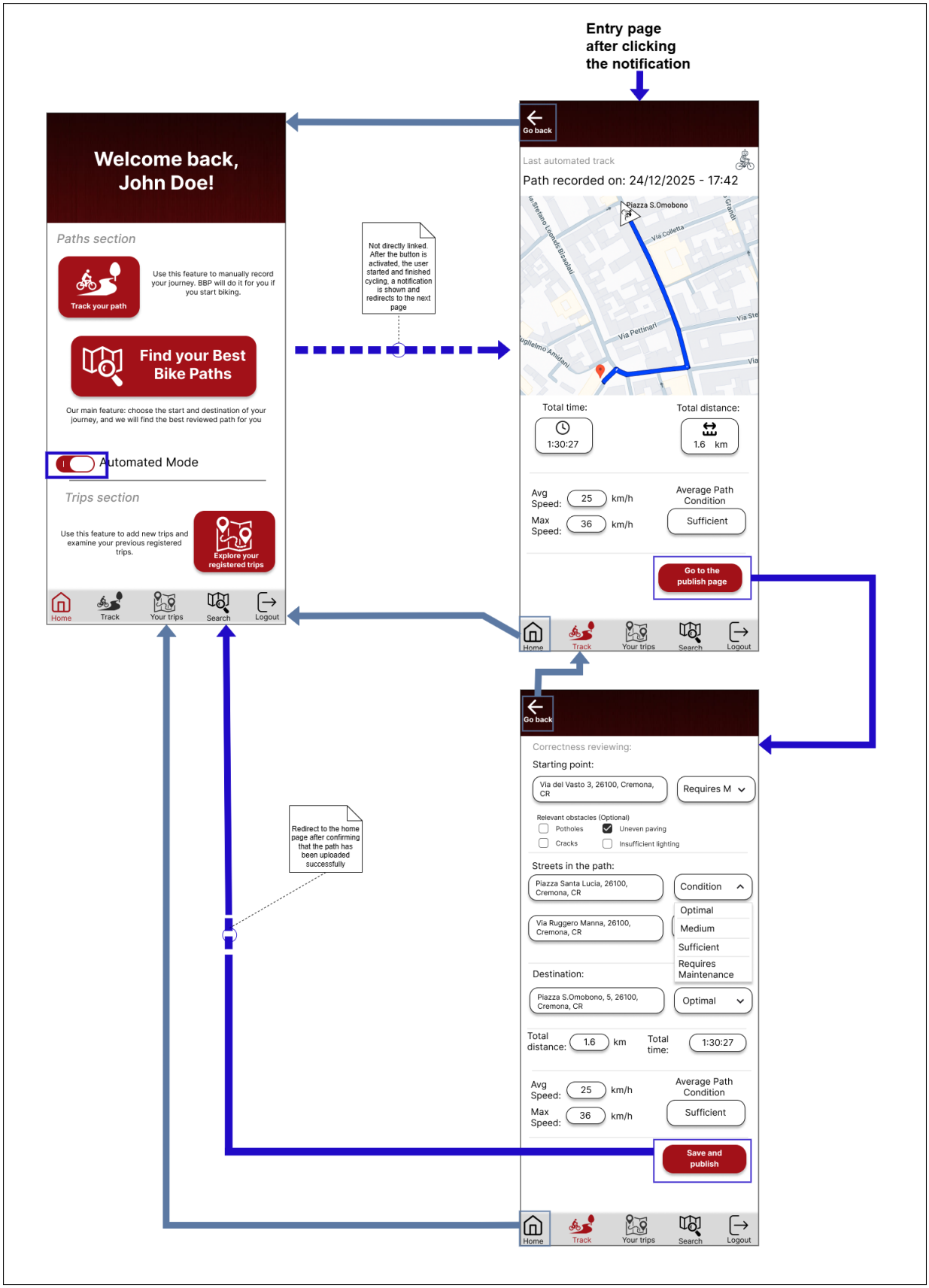


Figure 21: Interaction between the pages regarding the automated path tracking operation.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

A registered user can track a path by using the automated mode, in which the application will access the sensors in the user's smartphone. To do so, the "Automated Mode" button must be activated.
After the recording of a new path, a notification is displayed to the registered user, informing them that the recording was successful and it is ready to be reviewed. The registered user can see a summary of the recording by pressing the notification or opening the application. Here they can:

- move on to the next step (the actual edit and review) by pressing the "Go to the publish page" button;
- or can discard the recording by pressing either the "Go back" button or the Home icon in the Navigation Bar.

The edit and publish page is very similar to the manual one, in which the registered user can adjust the content to their liking. When the path needs to be registered into the system, the registered user can press the "Save and publish" button, successfully completing the operation. A notification is displayed to the registered user confirming the completion of the operation (not shown here for simplicity) and they are automatically redirected to the home page.
At any moment, they can abort the operation by pressing the Home icon in the Navigation Bar or move back to the review page by pressing the "Go back" button.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Trip recording and visualization

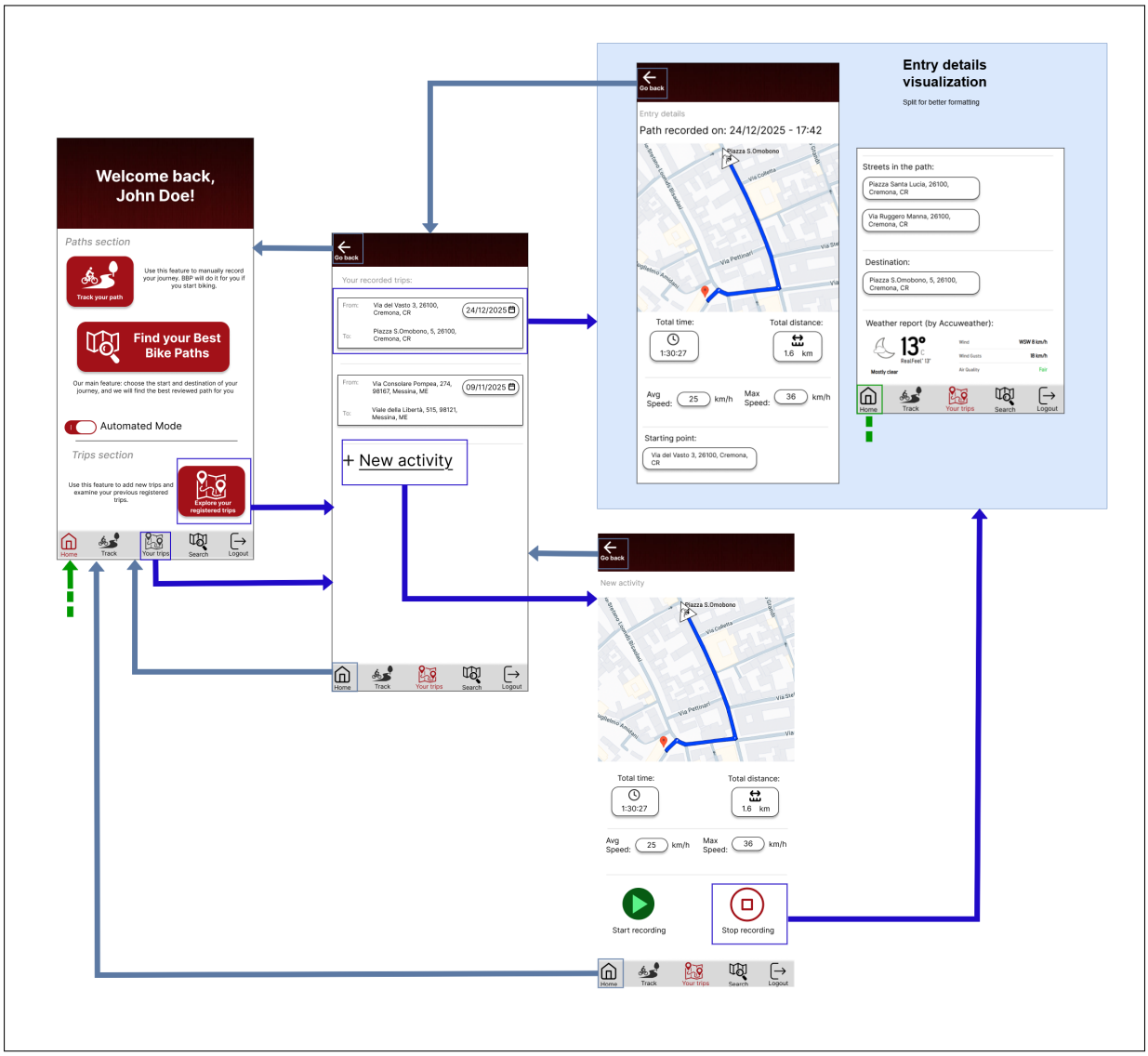


Figure 22: Interaction between the pages regarding the recording and visualization of trips (personal activities).

A registered user can track personal activities (or trips). To access the list of them, the registered user can press either the dedicated button "Explore your registered trips" or press the icon in the Navigation Bar.
Here, they are redirected into a new page, with the entire list of recorded activities. They can:

- visualize an entry detail by pressing on it. Everything important is showed here, included performance metrics and weather details;
- start the recording of a new entry by pressing on the text-button "New activity". They are redirected to a new page, in which the registered user can start, pause and stop the recording, while important metrics are visualized. After the registered user is ready to save the new activity, they can press the "Stop recording" button and, after a notification confirming the successful operation is displayed, they are redirected to the entry details page.

In each of the steps, the user can move back to the previous page or use the navigation bar.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Best Bike Paths search engine

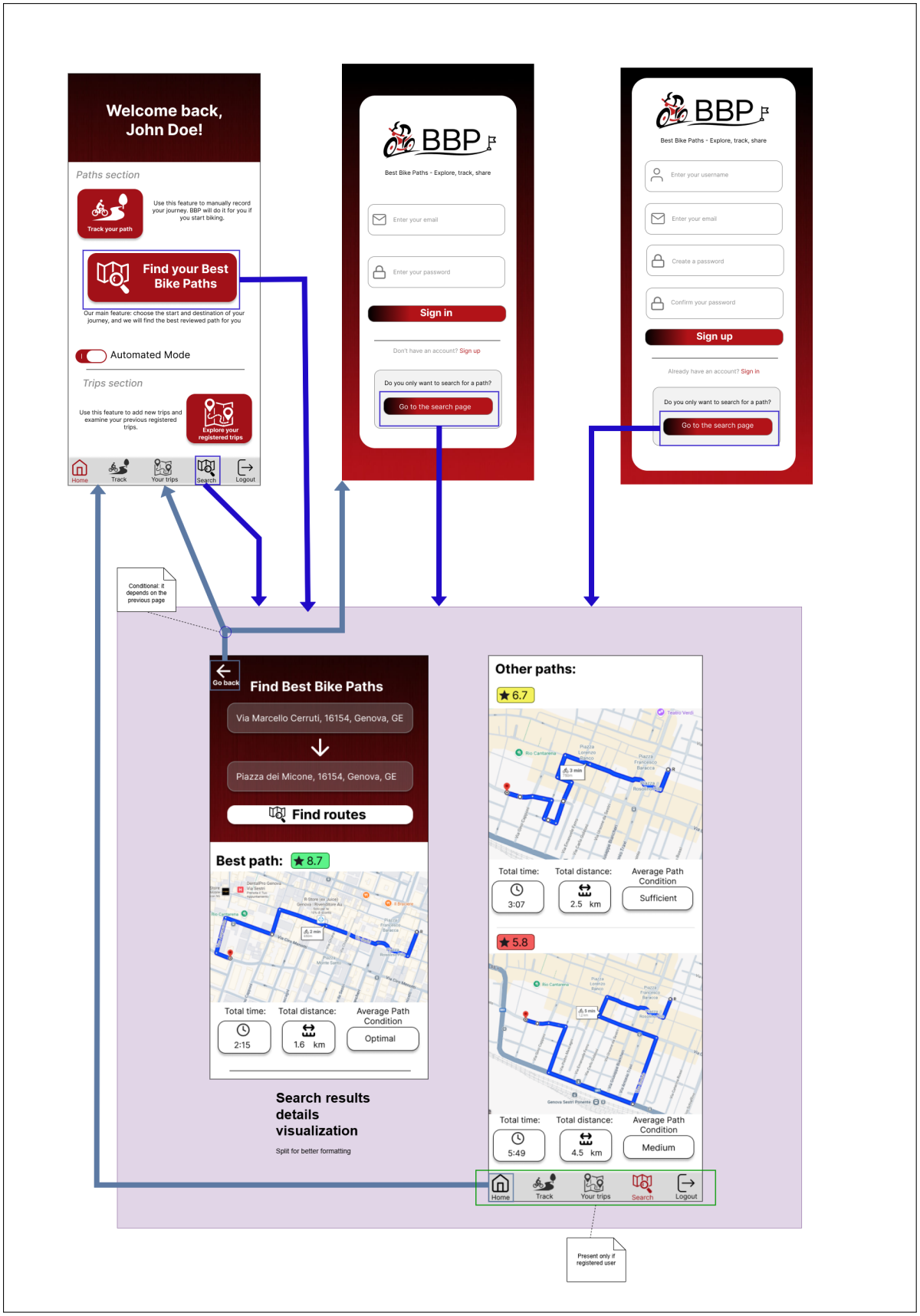


Figure 23: Interaction between the pages regarding the search of possible paths.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Any user (either registered or not) can search for the possible best bike path(s) using BBP system. To do so, they can access the search engine page by pressing either:

- the dedicated button in the home page (registered user only);
- the dedicated icon in the Navigation Bar (registered user only);
- the "Go to the search page" buttons in the Log In and Sign Up pages.

Here the user can, by inserting source and destination locations, search for the best possible bike path(s) between two points. If more than one is present in the database, the application will show them based on a calculated score. In any moment, they can go back to the previous page (either the home page or the log in page) and, for registered users only, use the navigation bar.

Final notes on Navigation Bar

In each page after the Log In, the Navigation Bar is present and can be used to access specific pages. This guarantees easy access and fast navigation through the pages.

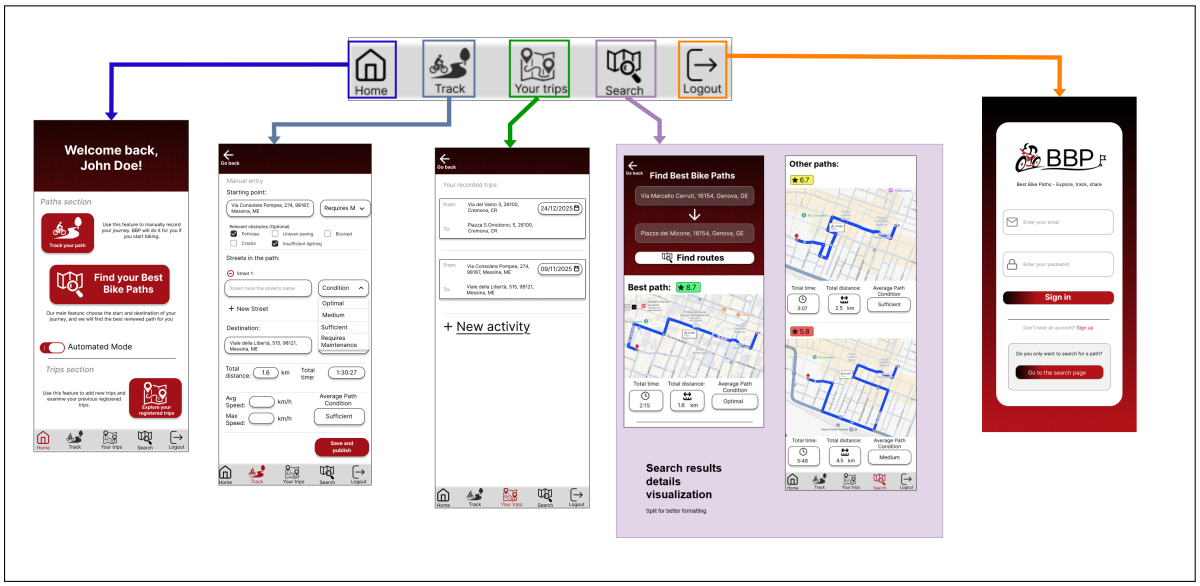


Figure 24: Navigation Bar links.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4 Requirements Traceability

4.1 Sign up process

Requirements	Components
R1: The system must allow users to register using a username, an email and a password.	User Service Component: • User Account Manager • User Authenticator Manager API Gateway Identity Provider Mobile Application Component User Database Email Delivery Service
R2: The system must verify that credentials are unique.	User Service Component: • User Account Manager • User Authenticator Manager API Gateway Identity Provider Mobile Application Component User Database
R3: The system must send a verification email to the user.	User Service Component: • User Account Manager • User Authenticator Manager Mobile Application Component API Gateway Email Delivery Service Identity Provider
R4: The system must let people verify their email after receiving the verification request.	User Service Component: • User Account Manager • User Authenticator Manager Email Delivery Service Identity Provider
R5: The system must activate accounts only if verified.	User Service Component: • User Account Manager • User Authenticator Manager User Database Identity Provider

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4.2 Login and Logout

Requirements	Components
R6: The system must allow registered users to log in using email and password.	Mobile Application Component Identity Provider
R7: The system must verify the correctness of the credentials	Mobile Application Component Identity Provider
R8: The system must allow registered users to log in only if the credentials are correct.	Mobile Application Component Identity Provider
R9: The system must allow registered users, that logged-in, to log out.	Mobile Application Component Identity Provider
R10: The system must delete user's session	User Service Component: • User Account Manager Identity Provider Mobile Application Component
R11: The system must allow only registered users to record trips and submit new bike path information.	Trip Service Component: • Trip Recorded Manager Mobile Application Component API Gateway
R12: The system must allow every user to access the system to browse paths.	User Service Component: • User Account Manager Path Service Component: • Path Search Engine – Best Path Finder – Information Merger Mobile Application Component API Gateway

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4.3 Permissions

Requirements	Components
R13: The system must ask registered users to give permissions for the sensors.	Mobile Application Component
R14: The system must read the sensors data only if the permissions are given by the registered user.	Mobile Application Component

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4.4 Trip Management

Requirements	Components
R15: The system must allow registered users to record their bike trips.	Trip Service Component: • Trip Recorded Manager Mobile Application Component API Gateway Trip Database
R16: The system must use sensors to track the biker's trip and their performance metrics.	Mobile Application Component
R17: The system must allow the registered user to save the trip recorded.	Trip Service Component: • Trip Recorded Manager Mobile Application Component API Gateway Trip Database
R18: The system must check the availability of the Weather Reporting Service (WRS) before requesting data.	Trip Service Component • Weather Information Querier • Trip Recording Manager Weather Report Service
R19: The system must ask the WRS for weather information regarding the trip considered.	Trip Service Component • Weather Information Querier • Trip Recording Manager Weather Report Service

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Requirements	Components
R20: The system must attach weather information to the trip entry, if available.	Trip Service Component • Weather Information Querier • Trip Recording Manager Weather Report Service Trip Database
R21: The system must show registered users a list of their recorded trips.	Trip Service Component • Recorded Trip Manager API Gateway Mobile Application Component Trip Database Server
R22: The system must allow registered users to see details of a specified recorded trip.	Trip Service Component • Recorded Trip Manager Mobile Application Component API Gateway Trip Database Server

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4.5 Bike Path Management- Manual mode

Requirements	Components
R23: The system must allow registered users to manually insert path information using a form.	Mobile Application Component
R24: The system must require the user to specify roads and intersections in the path.	Mobile Application Component Google Maps SDK
R25: The system must allow registered users to specify the condition of each road in the path.	Mobile Application Component
R26: The system must allow registered users to report additional issues (f.e. potholes).	Path Service Component: • Path Recording Manager API Gateway Mobile Application Component
R27: The system must save the path information in the central database.	Path Service Component: • Path Recording Manager Path Database Mobile Application Component API Gateway

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4.6 Bike Path Management- Automated mode

Requirements	Components
R28: The system must let registered users enable automated mode.	Mobile Application Component
R29: The system must let registered users disable automated mode.	Mobile Application Component
R30: The system must let registered users add path information via automated mode	Mobile Application Component
R31: The system must record data using sensors only if automated mode is enabled.	Mobile Application Component
R32: The system must recognize if the registered user is biking.	Mobile Application Component

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Requirements	Components
R33: The system must start recording the path only if the system recognizes that the registered user is biking.	Mobile Application Component
R34: The system must use the sensors' data to record the path.	Mobile Application Component
R35: The system must record intersections and road crossed by the registered user.	Mobile Application Component Google Maps SDK
R36: The system must recognize obstacles in the path (f.e. pothole) using sensors' data.	Mobile Application Component Google Maps SDK
R37: The system must recognize if the registered user stopped biking.	Mobile Application Component
R38: The system must stop recording the path if the system recognizes that the registered user stopped biking.	Mobile Application Component

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Requirements	Components
R39: The system must save the recorded bike path information in "to be reviewed" state after an automated mode recording.	Path Service Component: • Path Recording Manager API Gateway Mobile Application Component
R40: The system must notify the user that a path has been recorded this way.	Path Service Component: • Path Recording Manager API Gateway Mobile Application Component
R41: The system must require registered users to review and edit information collected via automated mode before publishing.	User Service Component: • User Account Manager Path Service Component: • Path Recording Manager Mobile Application Component
R42: The system must allow registered users to discard the information collected via automated mode.	Path Service Component: • Path Recording Manager API Gateway Path Database Mobile Application Component
R43: The system must allow registered users to save and publish the information recorded via automated mode.	Path Service Component: • Path Recording Manager API Gateway Path Database Mobile Application Component

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

4.7 Finding the Best Path

Requirements	Components
R44: The system must allow every user, regardless of their registration status, to browse for paths.	Path Service Component: • Path Search Engine – Best Path Finder – Information Merger Mobile Application Component API Gateway Path Database
R45: The system must allow the user to fill up a form regarding an origin and destination point.	Mobile Application Component
R46: The system must compute the best path(s) considering the specified points.	Path Service Component: • Path Search Engine – Best Path Finder – Information Merger
R47: The system must merge information from multiple users regarding a road, considering freshness of information and number of confirming users.	Path Service Component: • Path Search Engine – Best Path Finder – Information Merger Path Database
R48: The system must compute an average status for the specified path.	Path Service Component: • Path Search Engine – Best Path Finder – Information Merger

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

Requirements	**Components**
R49: The system must compute a score for the specified path, considering their status and the easy reachability.	Path Service Component: • Path Search Engine – Best Path Finder – Information Merger
R50: The system must show the user all the possible best paths in descendant score order, with their scores and metrics.	Path Service Component: • Path Search Engine – Best Path Finder – Information Merger Mobile Application Component API Gateway Path Database
R51: The system must show the user a map of the best path(s).	Mobile Application Component Google SDK Maps

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

5 Implementation, Integration and Test Plan

5.1 Implementation Plan

To accelerate the development process, the parallelization of system components is considered priority. The overall strategy is to adopt a bottom-up approach, focusing first on foundational components with the highest number of dependencies and subjecting them to rigorous testing.

This testing will initially consist of unit tests, as integrating this practice early in the development phase helps ensure a robust application infrastructure and reduces the need for extensive revisions later on. In the other words, unit testing enables early detection of bugs and errors, preventing situations in which issues require a comprehensive overhaul of the entire implementation.

In addiction, microservices implementations had to be taken into account. So, in order to test such services and to make sure that all units interact properly with each other, mocking APIs[6] have been considered. The isolation of failures, especially on early testing, is prioritized in order to, again, avoid bugs from affecting the entire system.

However, having microservices plus other components can increase the overall complexity, especially during implementation. But, as a good trade-off, they introduce a modularity that is way better than the monolithic approach during testing and future improvements of any kind.

5.2 Integration strategy

For this software architecture, microservices are essential. But, despite normal cases, where testing is required, external services are proved to be already reliable, such as Amazon Cognito or Maps API. The problem rises on the other microservices that utilize them. Before any actual implementation, we decide to adopt a testing strategy hybrid strategy between a top-down and bottom-up one, which allow clusters to be tested one by one, making them interact with mocks that simulate the behaviour of the services.

Take, for example, user recognition through Amazon Cognito. Or, in the case of the Trip cluster, the weather API. Obviously, the test can only be done by considering the mobile app as the driver, which will then be replaced later. Once it has been verified that the modules are working correctly, the load balancer is implemented at the infrastructure level to obtain an initial implementation of the system.

Additionally, the Discovery service must be mocked. The process begins by verifying the service registration mechanism. When a service instance starts, it should automatically register itself with the service registry, providing correct metadata such as service name, network address, and version, ensuring everything is correctly listed. Proper registration is essential, as a service that fails to register is effectively invisible to the rest of the system. This is also recommended over the retrieve, the heartbeat and the discovery processes.

After all services have been tested, all components should be integrated and see how the system responds to request loads. For example, to check performance, testing the complete system in a closed environment may be a great opportunity to see how the system handles, applying an End-to-end approach.

5.3 System testing

Once units testing has been completed and each component has been verified as operational and fully responsive, the final phase involves evaluating the system as a cohesive whole, especially taken account the microservices, e.g. weather's API, the Geo-localization services or even Amazon Cognito's API.

At this stage, comprehensive system-level testing must be conducted prior to deployment in order to identify and resolve any issues that could affect functionality or user experience. For this purpose, specific tests will be

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

executed to assess the overall performance, stability, security, and usability of the system, already mentioned on the RASD document.

A critical part of this evaluation will be performance testing, particularly under high workload conditions, to determine the system's ability to handle multiple concurrent requests, like users' devices connected at the same time to the network. Additionally, considering the potential size of the user base, usability testing will be of paramount importance. For instance, after the application has been developed, a heterogeneous group of users may be asked to complete surveys evaluating the system's intuitiveness and ease of use, in order to intervene on any issue raised by the testers.

Finally, another key factor is that external services are provided by external partners. So, any testing will be on the implementations of such components. However, contracts established with external service providers must guarantee sufficient computational capacity to handle our requests when users access or interact with the application.

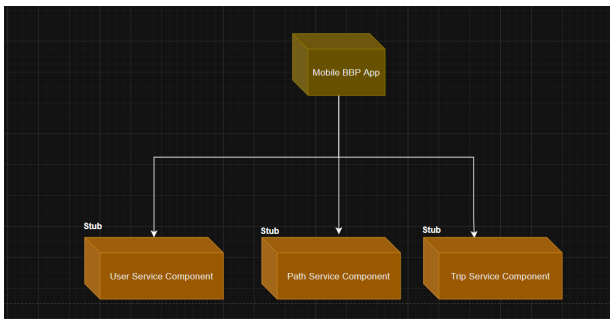


Figure 25: Testing BBP Mobile Application

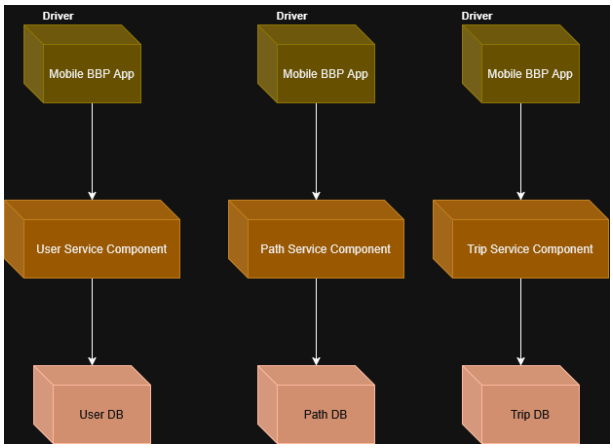


Figure 26: Unit Testing

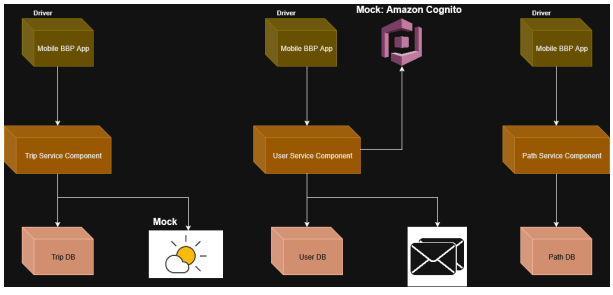


Figure 27: Mocking services in order to test each cluster

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

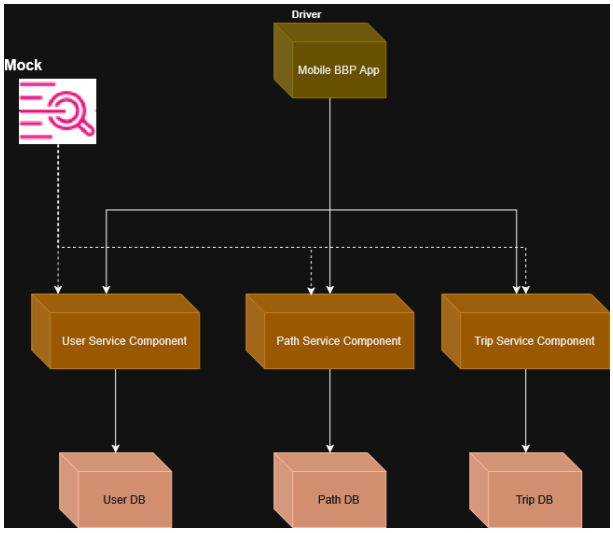


Figure 28: Mocking Search and Discovery Service

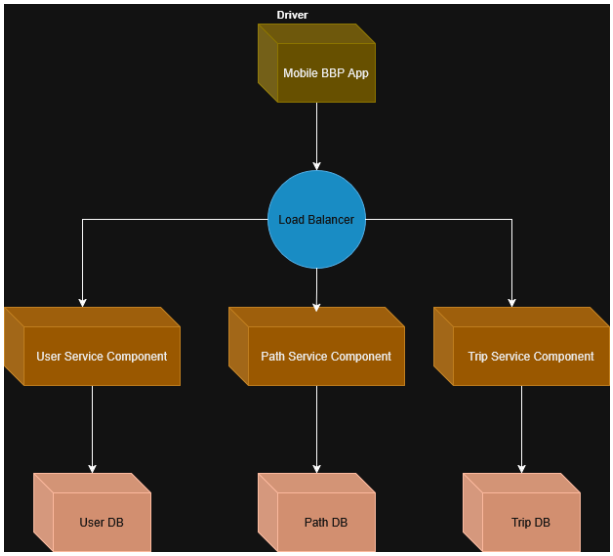


Figure 29: Testing at infrastructure level the load balancer

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

6 Effort Spent

The following table displays the number of hours each group member spent on the various sections of the document.

Group members	Chapter 1	Chapter 2	Chapter 3	Chapter 4	Chapter 5
Alessandro Barbera	1	16	6	1	1
Nicolas Bolanos	4	22	1	1	1
Massimiliano Nuzzo	1	8	2	7	8

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved

References

[1] Amazon (AWS Blog). Building fine-grained authorization using amazon cognito, api gateway, and iam, 2021.

[2] GeeksforGeeks. What are heartbeat messages?, 2024.

[3] GeeksforGeeks. Service discovery in distributed systems, 2025.

[4] GeeksforGeeks. What is api gateway?, 2025.

[5] Chris Richardson. Pattern: Microservice architecture, 2025.

[6] SpeedsScale. Top use cases for mock services in software, 2025.

Copyright © 2026, Alessandro Barbera, Nicolas Bolanos, Massimiliano Nuzzo – All rights reserved